Unity Graphics Programming - Volume 1

Contents

Preface

Translation and annotations by Claude

This book primarily explains techniques related to graphics programming in Unity.

The field of "graphics programming" is vast—even if you focus solely on shader techniques, many books have been published on the subject. This book contains articles on various topics selected according to the interests of the authors, but most of the content should produce visually striking results and be practical for creating your own effects.

The source code explained in each chapter is available at https://github.com/IndieVisualLab/UnityGraphicsProgramming, so you can run the examples locally while reading.

> **About This Translation**
>
> This English translation includes additional explanatory notes (like this one) to help clarify concepts for readers who may be new to graphics programming or Unity. The original Japanese text has been faithfully translated, with enhancements where they aid understanding.

Who This Book Is For

The difficulty level varies by chapter—depending on your existing knowledge, some content may feel too basic while other parts may seem quite challenging. We recommend reading the articles on topics that interest you, according to your current skill level.

For professional graphics programmers: We hope this book expands your toolkit of effects and techniques.

For students interested in visual coding: If you've experimented with Processing or openFrameworks but still find 3D computer graphics intimidating, we hope Unity can serve as an accessible entry point to discover the expressive power and development approaches of 3DCG.

About IndieVisualLab

IndieVisualLab is a circle (doujin group) founded by colleagues and former colleagues from the same company. At work, we use Unity for content programming of exhibition pieces—typically categorized as "media art." This represents a different flavor of Unity usage compared to game development.

Throughout this book, you may find knowledge scattered here and there that proves useful when utilizing Unity for exhibition and installation work.

Feedback and Requests

If you have feedback, questions, or requests about this book (such as "I'd like to read an explanation about X"), please contact us via the web form or email at lab.indievisual@gmail.com.

Note: The original feedback form link was for the Japanese edition. For questions about this translation or the code, please refer to the GitHub repository.

Chapter 2: Introduction to Compute Shaders

Original author: @XJINE Translation and annotations by Claude

---

Overview

This chapter provides a straightforward explanation of how to use Compute Shaders in Unity.

A **Compute Shader** uses the GPU to parallelize simple operations and execute massive amounts of calculations at high speed. While it delegates processing to the GPU, it differs from the traditional rendering pipeline—this is its key characteristic. In computer graphics, compute shaders are commonly used for things like expressing the movement of large numbers of particles.

Several later chapters in this book use compute shaders, so understanding the concepts here is essential for following along.

This chapter serves as your first stepping stone into learning compute shaders, using two simple samples to explain the basics. These samples don't cover everything about compute shaders, so you'll want to supplement your knowledge as needed.

Unity calls these “ComputeShaders,” but similar technologies include OpenCL, DirectCompute, and CUDA. The fundamental concepts are similar across all of these, and DirectCompute (DirectX) is particularly closely related. If you need more detailed information about the underlying architecture, looking into these technologies together would be helpful.

The sample code for this chapter is in the “SimpleComputeShader” folder at: https://github.com/IndieVisualLab/UnityGraphi

- **Why Compute Shaders Matter**

  Traditional GPU programming (vertex shaders, fragment shaders) is designed around the rendering pipeline—transforming vertices and coloring pixels. Compute shaders break free from this constraint. They let you use the GPU as a general-purpose parallel processor for *any* calculation, not just rendering.

  Think of it this way: a modern GPU has thousands of cores. A CPU might have 8-16. If your problem can be broken into thousands of independent pieces, compute shaders can be orders of magnitude faster than CPU code.

The Concepts: Kernels, Threads, and Groups

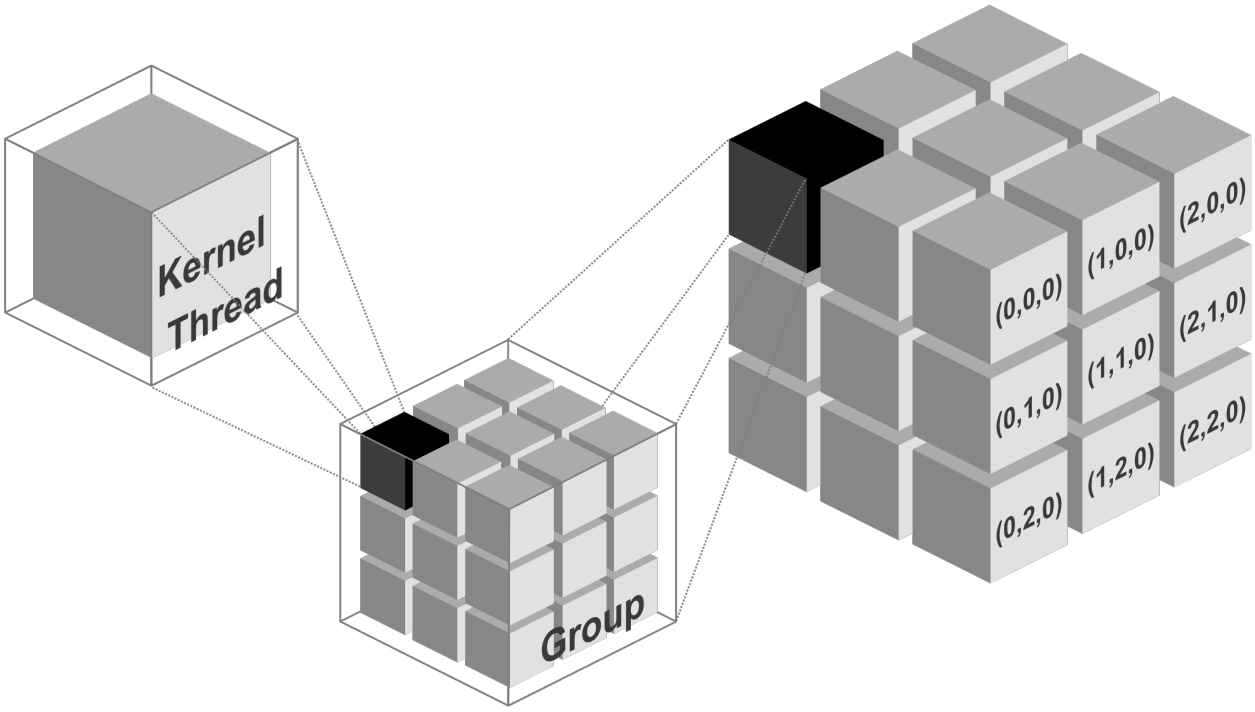


Figure: Visualization of Kernels, Threads, and Groups

Before diving into implementation details, we need to explain the concepts of **Kernel**, **Thread**, and **Group** that are used in compute shaders.

Kernel

A **kernel** refers to a single operation executed on the GPU, and in code, it’s treated as a single function (this corresponds to the general systems programming meaning of “kernel”).

- **Think of it simply:** A kernel is just a function that runs on the GPU instead of the CPU.

Thread

A **thread** is the unit that executes a kernel. One thread executes one kernel. With compute shaders, you can execute a kernel across multiple threads simultaneously in parallel.

Threads are specified in 3 dimensions: (x, y, z).

For example: - `(4, 1, 1)` means 4 × 1 × 1 = **4 threads** execute simultaneously - `(2, 2, 1)` means 2 × 2 × 1 = **4 threads** execute simultaneously

Both result in 4 threads, but depending on the situation, the second approach (specifying threads in 2 dimensions) can be more efficient. We'll explain this later. For now, just understand that thread counts are specified in 3 dimensions.

> **Why 3 dimensions?**
>
> The 3D specification maps naturally to many GPU problems: - **1D (x, 1, 1)**: Processing arrays, audio samples, particle lists - **2D (x, y, 1)**: Processing images, textures, height maps - **3D (x, y, z)**: Processing volumes, voxels, 3D simulations
>
> You'll almost always use 1D or 2D. 3D is for specialized volumetric work.

Group

A **group** is a unit that executes threads. The threads executed by a particular group are called **group threads**.

For example, if a group has `(4, 1, 1)` threads, and there are 2 such groups, each group has its own set of `(4, 1, 1)` threads.

Groups are also specified in 3 dimensions, just like threads. For example, if `(2, 1, 1)` groups execute a kernel with `(4, 4, 1)` threads: - Number of groups: 2 × 1 × 1 = **2 groups** - Each group has: 4 × 4 × 1 = **16 threads** - Total threads: 2 × 16 = **32 threads**

> **Why separate Groups and Threads?**
>
> This hierarchy exists because of GPU hardware architecture: - **Threads within a group** can share fast "shared memory" and synchronize with each other - **Threads in different groups** cannot easily communicate
>
> When you dispatch a compute shader, you specify the number of *groups*. The number of threads per group is defined in the shader itself. This separation gives you flexibility in how work is divided.

Sample 1: Retrieving Computed Results from the GPU

Sample 1, "SampleScene_Array," demonstrates how to execute arbitrary calculations in a compute shader and retrieve the results as an array.

This sample includes the following operations: - Process multiple pieces of data using a compute shader and retrieve the results - Implement multiple functions in a single compute shader and use them selectively - Pass values from a script (CPU) to the compute shader (GPU)

The execution result is debug output only, so please verify the behavior while reading the source code.

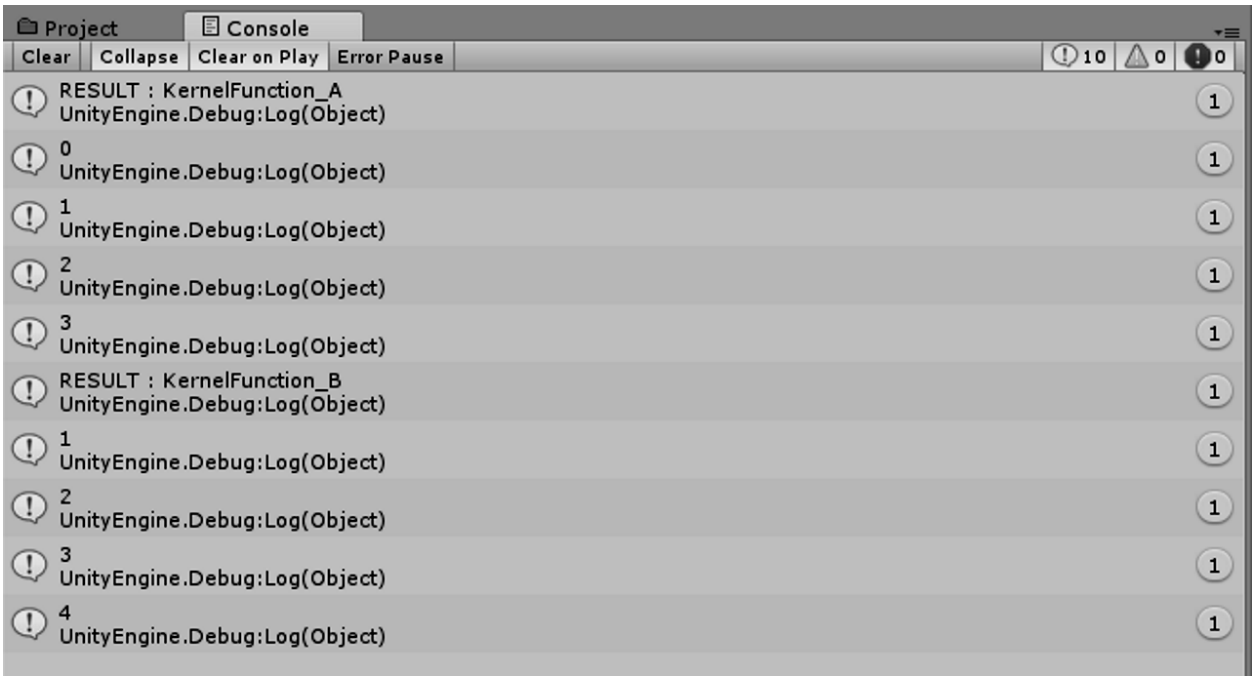


Figure: Sample 1 execution result

Compute Shader Implementation

Let's walk through the sample. The compute shader code is very short, so it's best to read through it entirely first. The basic structure consists of: function definitions, function implementations, buffers, and optionally, variables.

File: `SimpleComputeShader_Array.compute`

```
#pragma kernel KernelFunction_A
#pragma kernel KernelFunction_B

RWStructuredBuffer<int> intBuffer;
int intValue;

[numthreads(4, 1, 1)]
void KernelFunction_A(uint3 groupID : SV_GroupID,
                      uint3 groupThreadID : SV_GroupThreadID)
{
    intBuffer[groupThreadID.x] = groupThreadID.x * intValue;
}

[numthreads(4, 1, 1)]
void KernelFunction_B(uint3 groupID : SV_GroupID,
                      uint3 groupThreadID : SV_GroupThreadID)
{
    intBuffer[groupThreadID.x] += 1;
}
```

Notable features include the `numthreads` attribute and the `SV_GroupID` semantics, which we'll explain below.

- **Reading Compute Shader Syntax**

If you're coming from regular C# or C++, this syntax might look strange. Here's a quick decoder: - `#pragma kernel` — Declares a function as a GPU kernel (entry point) - `RWStructuredBuffer<int>` — A read-write array that lives on the GPU ("RW" = Read/Write) - `[numthreads(4, 1, 1)]` — An "attribute" that sets how many threads run this function - `uint3 groupID : SV_GroupID` — A parameter with a "semantic" that tells the GPU what value to inject

Kernel Definition

As explained earlier, a **kernel is a single operation executed on the GPU, treated as a single function in code**. You can implement multiple kernels in a single compute shader.

In this example, `KernelFunction_A` and `KernelFunction_B` are the kernels. Functions that should be treated as kernels are defined using `#pragma kernel`. This distinguishes kernels from other helper functions.

Each defined kernel is assigned a unique index for identification. Indices are assigned in order from top to bottom as 0, 1, … based on the order of `#pragma kernel` declarations.

Preparing Buffers and Variables

You need to create a **buffer area** to store results from compute shader execution. In the sample, `RWStructuredBuffer<int> intBuffer` serves this purpose.

- **Understanding RWStructuredBuffer**
  - `RW` = Read/Write (the GPU can both read from and write to this buffer)
  - `Structured` = It holds structured data (as opposed to raw bytes)
  - `Buffer<int>` = It's an array of integers

There's also `StructuredBuffer<T>` (read-only) for data you only need to read on the GPU.

If you want to pass arbitrary values from the script (CPU) side, you prepare variables just like in regular CPU programming. In this example, `intValue` serves this purpose, receiving its value from the script.

Specifying Thread Count with numthreads

The `numthreads` **attribute** specifies how many threads execute the kernel (function). Thread count is specified as (x, y, z). For example: - `(4, 1, 1)` = 4 × 1 × 1 = 4 threads execute the kernel - `(2, 2, 1)` = 2 × 2 × 1 = 4 threads execute the kernel

Both execute 4 threads, but we'll discuss the difference and when to use each approach later.

Kernel (Function) Parameters

Kernel parameters have constraints and offer extremely limited flexibility compared to regular CPU programming.

The values following the parameters are called **semantics**. In this example, we've set `groupID : SV_GroupID` and `groupThreadID : SV_GroupThreadID`. Semantics indicate what kind of value the parameter holds, and you cannot change them to other names.

While you can freely define parameter names (variable names), you must set one of the semantics defined for compute shaders. In other words, you **cannot** define arbitrary typed parameters for reference within the kernel—you must choose from a limited set of predefined semantics.

- `SV_GroupID` — Indicates which group (x, y, z) the thread executing this kernel belongs to
- `SV_GroupThreadID` — Indicates which thread number (x, y, z) within the group is executing this kernel

For example, when executing `(4, 4, 1)` groups with `(2, 2, 1)` threads: - `SV_GroupID` returns values in the range (0–3, 0–3, 0) - `SV_GroupThreadID` returns values in the range (0–1, 0–1, 0)

Common Compute Shader Semantics

Semantic	Meaning
`SV_GroupID`	Which group am I in? (0 to numGroups-1)
`SV_GroupThreadID`	Which thread am I within my group? (0 to numthreads-1)
`SV_DispatchThreadID`	My global thread ID across ALL threads (most commonly used!)
`SV_GroupIndex`	Flattened 1D index within my group

`SV_DispatchThreadID` is usually what you want—it gives you a unique ID for each thread across the entire dispatch.

There are other `SV_` semantics you can use, but we'll skip explaining them here. It's better to review them after understanding how compute shaders work.

- SV_GroupID - Microsoft Developer Network
  - https://msdn.microsoft.com/en-us/library/ee422449(v=vs.85).aspx
  - You can check different SV~ semantics and their values here.

Kernel (Function) Processing Content

The sample assigns thread numbers sequentially to the prepared buffer. `groupThreadID` receives the thread number within a group. Since this kernel executes with `(4, 1, 1)` threads, `groupThreadID` receives values (0–3, 0, 0).

```
[numthreads(4, 1, 1)]
void KernelFunction_A(uint3 groupID : SV_GroupID,
                      uint3 groupThreadID : SV_GroupThreadID)
{
    intBuffer[groupThreadID.x] = groupThreadID.x * intValue;
}
```

This sample executes these threads in `(1, 1, 1)` groups (from the script, explained later). This means we execute just 1 group, and that group contains 4 × 1 × 1 threads. Consequently, `groupThreadID.x` receives values 0–3.

What's happening here, step by step:

1. We dispatch 1 group
2. That group has 4 threads (numthreads is 4,1,1)
3. All 4 threads run KernelFunction_A **simultaneously**
4. Thread 0 sets `intBuffer[0] = 0 * 1 = 0`
5. Thread 1 sets `intBuffer[1] = 1 * 1 = 1`
6. Thread 2 sets `intBuffer[2] = 2 * 1 = 2`
7. Thread 3 sets `intBuffer[3] = 3 * 1 = 3`

All four assignments happen at the same time! That's the power of parallelism.

Note: This example doesn't use `groupID`, but it receives the group count specified in 3 dimensions just like threads. Try assigning it to verify compute shader behavior.

Executing the Compute Shader from a Script

We execute the implemented compute shader from a script. The script side generally requires:

- Reference to the compute shader | `computeShader`
- Index of the kernel to execute | `kernelIndex_KernelFunction_A`, `kernelIndex_KernelFunction_B`
- Buffer to store compute shader execution results | `intComputeBuffer`

File: `SimpleComputeShader_Array.cs`

```
public ComputeShader computeShader;
int kernelIndex_KernelFunction_A;
int kernelIndex_KernelFunction_B;
ComputeBuffer intComputeBuffer;

void Start()
{
    // (1) Store kernel indices
    this.kernelIndex_KernelFunction_A
        = this.computeShader.FindKernel("KernelFunction_A");
    this.kernelIndex_KernelFunction_B
        = this.computeShader.FindKernel("KernelFunction_B");

    // (2) Set up the ComputeBuffer to store calculation results
    // A buffer with the same type and name must be defined in the ComputeShader
    this.intComputeBuffer = new ComputeBuffer(4, sizeof(int));
    this.computeShader.SetBuffer
        (this.kernelIndex_KernelFunction_A,
         "intBuffer", this.intComputeBuffer);

    // (3) Pass parameters to the ComputeShader if needed
    this.computeShader.SetInt("intValue", 1);
    ...
}
```

Getting the Kernel Index

To execute a kernel, you need index information to identify it. Indices are assigned from 0, 1, … in order of `#pragma kernel` definitions from top to bottom, but using the `FindKernel` function from the script side is recommended.

```
this.kernelIndex_KernelFunction_A
    = this.computeShader.FindKernel("KernelFunction_A");

this.kernelIndex_KernelFunction_B
    = this.computeShader.FindKernel("KernelFunction_B");
```

Creating a Buffer to Store Computation Results

We prepare a buffer area on the CPU side to store computation results from the compute shader (GPU). In Unity, this is defined as `ComputeBuffer`.

```
this.intComputeBuffer = new ComputeBuffer(4, sizeof(int));
this.computeShader.SetBuffer
    (this.kernelIndex_KernelFunction_A, "intBuffer", this.intComputeBuffer);
```

`ComputeBuffer` is initialized by specifying (1) the size of the area to allocate and (2) the size per unit of data to store. Here, space for 4 ints is allocated because the compute shader execution result will be stored as `int[4]`. Adjust the size as needed.

Next, we specify (1) which kernel's execution, (2) which GPU buffer it uses, and (3) which CPU buffer it corresponds to.

In this example: (1) when `KernelFunction_A` executes, (2) the buffer area named `intBuffer` (3) corresponds to `intComputeBuffer`.

> **Understanding the Buffer Binding**
>
> This is a crucial concept: the GPU and CPU have **separate memory**. The `ComputeBuffer` creates memory on the GPU, and `SetBuffer` tells the compute shader "when you reference `intBuffer` in your code, use this specific GPU memory."
>
> The name `"intBuffer"` must exactly match the variable name in your .compute file!

Passing Values from Script to Compute Shader

```
this.computeShader.SetInt("intValue", 1);
```

Depending on your processing needs, you may want to pass values from the script (CPU) side to the compute shader (GPU) side for reference. Most value types can be set to variables in the compute shader using `ComputeShader.Set~` methods. The parameter variable name passed as an argument must match the variable name defined in the compute shader. In this example, we pass 1 to `intValue`.

> **Available Set Methods**
>
> - `SetInt`, `SetFloat`, `SetBool` — Single values
> - `SetVector` — Vector4 (also works for Vector2, Vector3)
> - `SetMatrix` — 4x4 matrix
> - `SetBuffer` — ComputeBuffer (arrays)
> - `SetTexture` — RenderTexture (images)
>
> All of these require the name to match exactly with the variable in your shader.

Executing the Compute Shader

Kernels defined (implemented) in the compute shader are executed using the `ComputeShader.Dispatch` method. It executes the kernel at the specified index with the specified number of groups. Group count is specified as X × Y × Z. In this sample, it's 1 × 1 × 1 = 1 group.

```
this.computeShader.Dispatch
    (this.kernelIndex_KernelFunction_A, 1, 1, 1);

int[] result = new int[4];

this.intComputeBuffer.GetData(result);

for (int i = 0; i < 4; i++)
{
    Debug.Log(result[i]);
}
```

Compute shader (kernel) execution results are retrieved using `ComputeBuffer.GetData`.

> **Performance Warning**

> `GetData` is **slow**! It forces the CPU to wait for the GPU to finish, then copies data from GPU memory to CPU memory. In real applications: - Avoid calling `GetData` every frame if possible - Use `AsyncGPUReadback` for non-blocking reads (Unity 2018.1+) - Often you don't need to read back at all—just use the buffer directly in rendering

Verifying Execution Results (A)

Let's review the compute shader implementation again. This sample executes the following kernel with 1 × 1 × 1 = 1 group. Threads are 4 × 1 × 1 = 4 threads. Also, `intValue` receives 1 from the script.

```
[numthreads(4, 1, 1)]
void KernelFunction_A(uint3 groupID : SV_GroupID,
                      uint3 groupThreadID : SV_GroupThreadID)
{
    intBuffer[groupThreadID.x] = groupThreadID.x * intValue;
}
```

`groupThreadID (SV_GroupThreadID)` contains the value indicating which thread within the group is currently executing this kernel. In this example, it contains (0–3, 0, 0). Therefore, `groupThreadID.x` is 0–3.

This means `intBuffer[0] = 0` through `intBuffer[3] = 3` are executed in parallel.

Executing a Different Kernel (B)

To execute a different kernel implemented in the same compute shader, specify that kernel's index. In this example, we execute `KernelFunction_B` after `KernelFunction_A`. Furthermore, we reuse the buffer area used by `KernelFunction_A` for `KernelFunction_B`.

```
this.computeShader.SetBuffer
    (this.kernelIndex_KernelFunction_B, "intBuffer", this.intComputeBuffer);

this.computeShader.Dispatch(this.kernelIndex_KernelFunction_B, 1, 1, 1);

this.intComputeBuffer.GetData(result);

for (int i = 0; i < 4; i++)
{
    Debug.Log(result[i]);
}
```

Verifying Execution Results (B)

`KernelFunction_B` executes the following code. Note that `intBuffer` continues to reference the same buffer used by `KernelFunction_A`.

```
RWStructuredBuffer<int> intBuffer;

[numthreads(4, 1, 1)]
void KernelFunction_B
    (uint3 groupID : SV_GroupID, uint3 groupThreadID : SV_GroupThreadID)
{
    intBuffer[groupThreadID.x] += 1;
}
```

In this sample, `intBuffer` has been assigned 0–3 sequentially by `KernelFunction_A`. Therefore, after executing `KernelFunction_B`, verify that the values become 1–4.

Releasing the Buffer

When you're done using a `ComputeBuffer`, you must explicitly release it.

```
this.intComputeBuffer.Release();
```

- **Memory Management**

  Unlike regular C# objects, `ComputeBuffer` allocates GPU memory that is **not** automatically garbage collected. If you forget to call `Release()`, you'll leak GPU memory!

  Best practice: Release buffers in `OnDestroy()`:

```
void OnDestroy()
{
    if (intComputeBuffer != null)
        intComputeBuffer.Release();
}
```

Issues Not Addressed in Sample 1

This sample doesn't explain the intent behind specifying multi-dimensional threads or groups. For example, both `(4, 1, 1)` threads and `(2, 2, 1)` threads execute 4 threads, but there's meaning in distinguishing between them. This is explained in Sample 2 that follows.

Sample 2: Turning GPU Computation Results into a Texture

Sample 2, "SampleScene_Texture," retrieves compute shader calculation results as a texture.

This sample includes the following operations: - Use a compute shader to write information to a texture - Effectively utilize multi-dimensional (2D) threads

The execution result of Sample 2 is as follows. It generates textures that gradient horizontally and vertically.

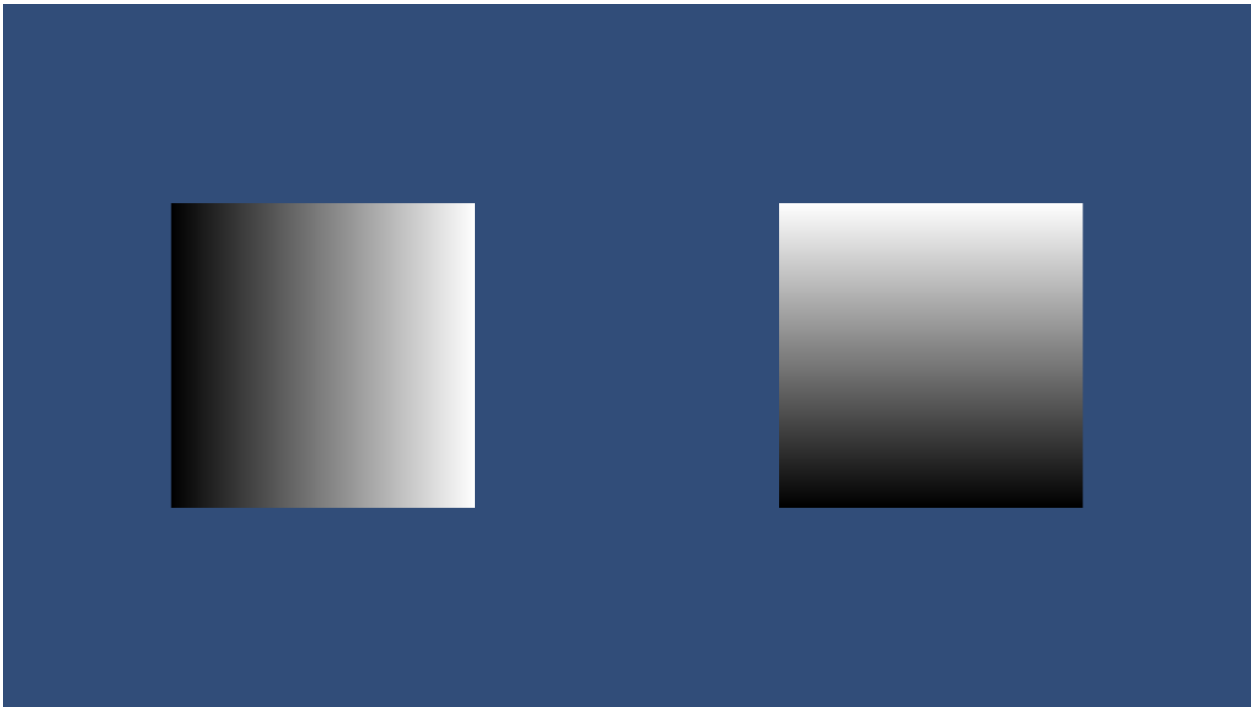


Figure: Sample 2 execution result

Kernel Implementation

Please refer to the sample for the full implementation. This sample executes roughly the following code in the compute shader. Note that the kernel executes with multi-dimensional threads: `(8, 8, 1)` means 8 × 8 × 1 = 64 threads per group.

Also, a major change is that the storage destination for computation results is `RWTexture2D<float4>`.

File: `SimpleComputeShader_Texture.compute`

```
RWTexture2D<float4> textureBuffer;

[numthreads(8, 8, 1)]
void KernelFunction_A(uint3 dispatchThreadID : SV_DispatchThreadID)
{
    float width, height;
    textureBuffer.GetDimensions(width, height);

    textureBuffer[dispatchThreadID.xy]
        = float4(dispatchThreadID.x / width,
                 dispatchThreadID.x / width,
                 dispatchThreadID.x / width,
                 1);
}
```

Understanding RWTexture2D

- `RWTexture2D` — A 2D texture the GPU can write to
- `<float4>` — Each pixel stores 4 floats (R, G, B, A)
- `textureBuffer[xy]` — Access pixel at coordinates (x, y)

This is perfect for image processing, simulations, or any 2D data.

The Special Parameter SV_DispatchThreadID

In Sample 1, we didn't use the `SV_DispatchThreadID` semantic. It's somewhat complex, but it indicates **"where a thread executing a kernel is positioned among all threads (x, y, z)"**.

`SV_DispatchThreadID` is calculated as: `SV_GroupID × numthreads + SV_GroupThreadID`

Where `SV_GroupID` indicates a group in (x, y, z), and `SV_GroupThreadID` indicates a thread within a group in (x, y, z).

Why SV_DispatchThreadID is Usually What You Want

In most cases, you don't care about groups—you just want to know "which piece of data am I processing?"

`SV_DispatchThreadID` gives you exactly that: a unique (x, y, z) coordinate across your entire dataset.

For a 512×512 texture with 8×8 thread groups: - You dispatch 64×64 groups - Each thread gets a unique `dispatchThreadID` from (0,0,0) to (511,511,0) - Thread at (100, 200, 0) processes pixel (100, 200)

Concrete Calculation Example (1)

For example, let's say we execute a kernel with `(2, 2, 1)` groups and `(4, 1, 1)` threads. One of those kernels is executed in group `(0, 1, 0)`, thread `(2, 0, 0)`.

`SV_DispatchThreadID` = `(0, 1, 0)` × `(4, 1, 1)` + `(2, 0, 0)` = `(0, 1, 0)` + `(2, 0, 0)` = `(2, 1, 0)`

Concrete Calculation Example (2)

Now let's consider the maximum value. When executing with `(2, 2, 1)` groups and `(4, 1, 1)` threads, the last thread is in group `(1, 1, 0)`, thread `(3, 0, 0)`.

`SV_DispatchThreadID` = `(1, 1, 0)` × `(4, 1, 1)` + `(3, 0, 0)` = `(4, 1, 0)` + `(3, 0, 0)` = `(7, 1, 0)`

Writing Values to a Texture (Pixels)

From here, it's difficult to explain in chronological order, so please review the entire sample while following along.

In Sample 2, `dispatchThreadID.xy` is set up (through groups and threads) to represent all pixels on the texture. Since groups are set on the script side, you need to verify across both the script and compute shader.

```
textureBuffer[dispatchThreadID.xy]
    = float4(dispatchThreadID.x / width,
             dispatchThreadID.x / width,
             dispatchThreadID.x / width,
             1);
```

In this sample, we've prepared a 512×512 texture. When `dispatchThreadID.x` ranges from 0–511, `dispatchThreadID.x / width` ranges from 0 to ~0.998. This means as the `dispatchThreadID.xy` value (= pixel coordinate) increases, pixels are painted from black to white.

> **Note** Textures are composed of RGBA channels, each set from 0 to 1. When all are 0, it's completely black; when all are 1, it's completely white.

Preparing the Texture

The following is an explanation of the script-side implementation. In Sample 1, we prepared an array buffer to store compute shader calculation results. In Sample 2, we prepare a texture instead.

File: `SimpleComputeShader_Texture.cs`

```
RenderTexture renderTexture_A;
...
void Start()
{
    this.renderTexture_A = new RenderTexture
        (512, 512, 0, RenderTextureFormat.ARGB32);
    this.renderTexture_A.enableRandomWrite = true;
    this.renderTexture_A.Create();
    ...
}
```

Initialize a RenderTexture specifying resolution and format. Note that `RenderTexture.enableRandomWrite` must be enabled to allow writing to the texture.

Why enableRandomWrite?

By default, RenderTextures are optimized for the graphics pipeline (sequential writes during rendering). `enableRandomWrite = true` tells Unity "this texture will be written to from compute shaders, where any thread might write to any pixel." This enables the necessary GPU memory access patterns.

- RenderTexture.enableRandomWrite - Unity
  - https://docs.unity3d.com/ScriptReference/RenderTexture-enableRandomWrite.html

Getting Thread Count

Just as you can get a kernel's index, you can also get how many threads a kernel executes with (thread size).

```
void Start()
{
    ...
    uint threadSizeX, threadSizeY, threadSizeZ;

    this.computeShader.GetKernelThreadGroupSizes
        (this.kernelIndex_KernelFunction_A,
         out threadSizeX, out threadSizeY, out threadSizeZ);
    ...
}
```

Executing the Kernel

Execute processing with the `Dispatch` method. Pay attention to how the group count is specified. In this example, group count is calculated as "texture horizontal (vertical) resolution / horizontal (vertical) thread count."

Considering the horizontal direction: texture resolution is 512, thread count is 8, so horizontal group count is 512 / 8 = 64. Similarly, vertical is also 64. Therefore, total group count is 64 × 64 = 4,096.

```
void Update()
{
    this.computeShader.Dispatch
        (this.kernelIndex_KernelFunction_A,
         this.renderTexture_A.width / this.kernelThreadSize_KernelFunction_A.x,
         this.renderTexture_A.height / this.kernelThreadSize_KernelFunction_A.y,
         this.kernelThreadSize_KernelFunction_A.z);

    plane_A.GetComponent<Renderer>()
        .material.mainTexture = this.renderTexture_A;
}
```

In other words, each group processes 8 × 8 × 1 = 64 (= thread count) pixels. There are 4,096 groups, so 4,096 × 64 = 262,144 pixels are processed. The image is 512 × 512 = 262,144 pixels, so exactly all pixels are processed in parallel.

The Key Formula

`Total Threads = Groups × Threads per Group`

For textures, you typically want:

```
groupsX = textureWidth / numthreads.x
groupsY = textureHeight / numthreads.y
```

This ensures every pixel gets exactly one thread.

Executing Different Kernels The other kernel fills using the y coordinate instead of x. Note that values close to 0 (black colors) appear at the bottom. When manipulating textures, you sometimes need to consider the origin position.

Benefits of Multi-dimensional Threads and Groups

As in Sample 2, when you need multi-dimensional results or multi-dimensional calculations, multi-dimensional threads and groups work effectively. If you tried to process Sample 2 with 1-dimensional threads, you'd need to arbitrarily calculate vertical pixel coordinates.

> **Note** If you actually try to implement this, you'll see it requires calculating stride in image processing terms—for example, with a 512×512 image, the 513th pixel would be at coordinate (0, 1).

It's better to reduce calculation count, and complexity increases with more advanced processing. When designing processing using compute shaders, it's good to consider whether you can effectively utilize multiple dimensions.

Supplementary Information for Further Learning

This chapter has covered introductory compute shader information through sample explanations, but here's some supplementary information needed for continued learning.

GPU Architecture and Basic Structure

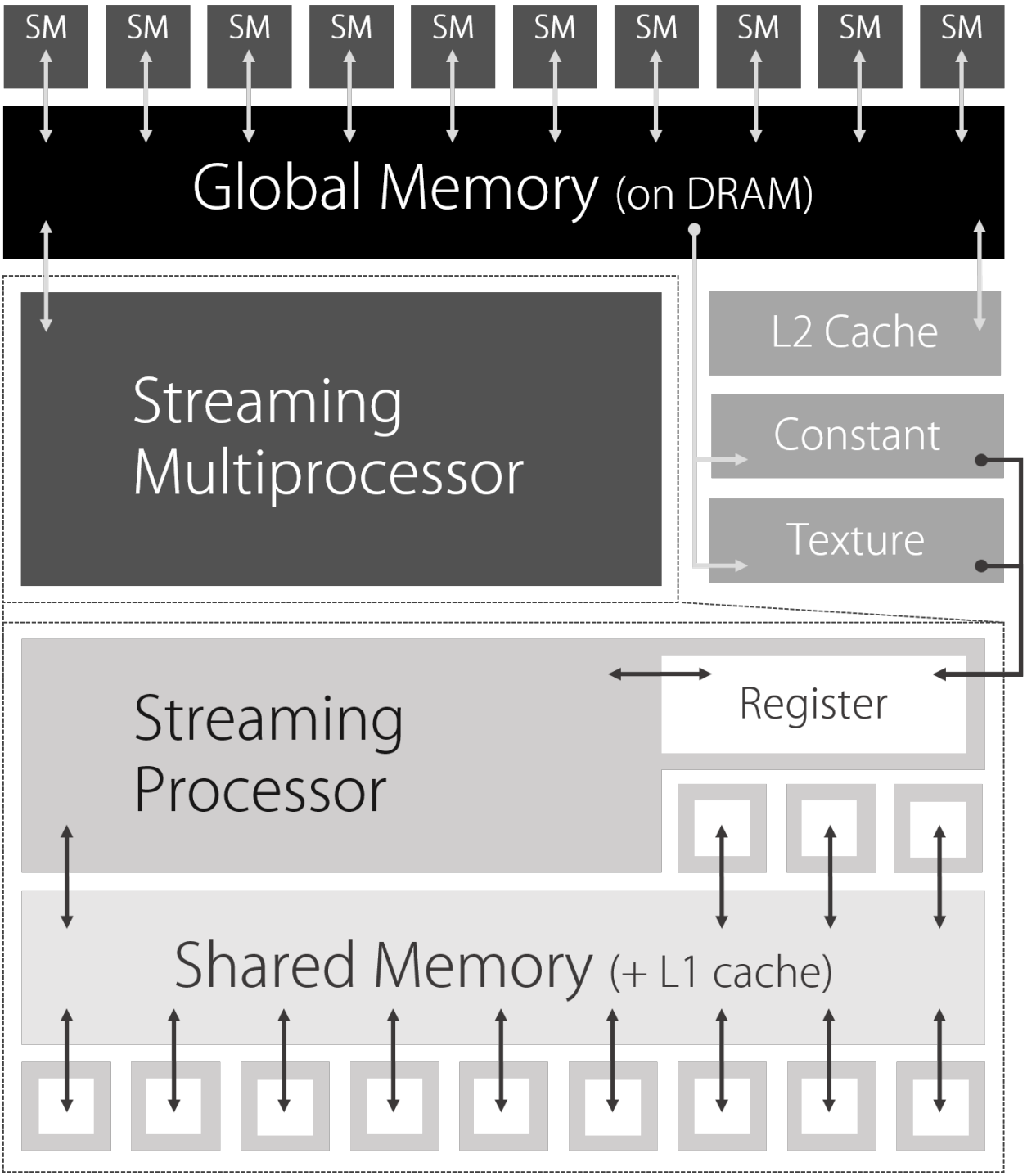


Figure: Image of GPU architecture

Basic knowledge about GPU architecture and structure will help optimize your implementations when using compute shaders, so let's briefly introduce it here.

A GPU contains many **Streaming Multiprocessors (SMs)** that share and parallelize assigned processing.

Each SM contains multiple smaller **Streaming Processors (SPs)**, and the processing assigned to an SM is calculated by the SPs.

SMs have **registers** and **shared memory**, which can be read and written to faster than **global memory (DRAM memory)**. Registers are used for local variables referenced only within functions, and shared memory can be referenced and written to by all SPs belonging to the same SM.

In other words, the ideal is to understand the maximum size and scope of each memory type and implement in a way that reads and writes memory quickly without waste.

- **The Memory Hierarchy (from fastest to slowest)**

  1. **Registers** — Fastest, but limited, private to each thread
  2. **Shared Memory** — Fast, shared within a group (marked with `groupshared`)
  3. **L1/L2 Cache** — Automatic caching of global memory
  4. **Global Memory** — Large but slow, accessible by all threads

  Real optimization often involves loading data into shared memory, processing it there, then writing results back to global memory.

For example, shared memory, which probably requires the most consideration, is defined using the storage-class modifier `groupshared`. We'll omit concrete examples here as this is introductory, but remember this as a technique and term needed for optimization and use it in your continued learning.

- Variable Syntax - Microsoft Developer Network
  - https://msdn.microsoft.com/en-us/library/bb509706(v=vs.85).aspx

Registers Memory placed closest to the SP, with the fastest access. Composed in 4-byte units; kernel (function) scope variables are placed here. Independent per thread and cannot be shared.

Shared Memory Memory placed in the SM, managed together with L1 cache. Can be shared among SPs (= threads) within the same SM, and can be accessed sufficiently fast.

Global Memory Memory on DRAM, not on the GPU. Reference is slow because it's distant from GPU processors. On the other hand, capacity is large, and data can be read and written by all threads.

Local Memory Memory on DRAM, not on the GPU; stores data that doesn't fit in registers. Reference is slow because it's distant from GPU processors.

Texture Memory Dedicated memory for texture data; handles global memory specifically for textures.

Constant Memory Read-only memory used for storing kernel (function) arguments and constants. Has a dedicated cache, so it can be referenced faster than global memory.

Hints for Efficient Thread Count Specification

If total thread count is larger than the actual data count you want to process, threads that execute meaninglessly (or don't process anything) will occur, which is inefficient. Design total thread count to match the data count you want to process as much as possible.

- **Handling Non-divisible Sizes**

  What if your texture is 500×500 but your threads are 8×8? 500 doesn't divide evenly by 8!

  Common solutions: 1. **Pad your texture** to a multiple of 8 (512×512) 2. **Round up groups** and add bounds checking in the shader: `hlsl if (dispatchThreadID.x >= width || dispatchThreadID.y >= height) return; // Early exit for out-of-bounds threads`

Current Spec Limitations

Here are the upper limits of current specs at the time of writing. Please note these may not be the latest. However, you'll need to implement while considering these kinds of limitations.

- Compute Shader Overview - Microsoft Developer Network
  - https://msdn.microsoft.com/en-us/library/ff476331(v=vs.85).aspx

Thread and Group Count We didn't mention the limits of thread and group counts during the explanation because they change depending on shader model (version). The number that can be parallelized is expected to increase going forward.

Shader Model cs_4_x: - Maximum Z value is 1 - Maximum X × Y × Z is 768

Shader Model cs_5_0: - Maximum Z value is 64 - Maximum X × Y × Z is 1024

Group limits are 65535 each for (x, y, z).

Modern Unity (Unity 6)

Unity 6 typically targets Shader Model 5.0 or higher. You can safely use up to 1024 threads per group. Common configurations: - `[numthreads(256, 1, 1)]` — For 1D data (particles, arrays) - `[numthreads(8, 8, 1)]` or `[numthreads(16, 16, 1)]` — For 2D data (images) - `[numthreads(8, 8, 8)]` — For 3D data (volumes)

Memory Areas Shared memory upper limit is 16 KB per group. The size of shared memory a single thread can write to is limited to 256 bytes per unit.

References

Other references for this chapter are as follows:

- Chapter 5: GPU Structure - Japan GPU Computing Partnership - http://www.gdep.jp/page/view/252
- Getting Started with CUDA on Windows - NVIDIA Japan - http://on-demand.gputechconf.com/gtc/2013/jp/sessions/8

Summary: Key Takeaways

What You Should Remember

1. **Compute shaders run functions (kernels) on the GPU in parallel**
2. **The hierarchy: Groups Threads Kernel execution**
   - You dispatch groups
   - Each group contains threads (defined by `numthreads`)
   - Each thread runs the kernel once
3. **Use `SV_DispatchThreadID` to know which data element you're processing**
4. **Match dimensions to your problem:**
   - 1D threads for arrays/particles
   - 2D threads for images/textures
5. **Memory management is manual:** Always `Release()` your `ComputeBuffers`!
6. **The CPU GPU boundary is slow:** Minimize `GetData` calls; keep data on the GPU when possible

---

Next chapter: GPU Boids Simulation — see compute shaders in action with thousands of autonomous agents!

Chapter 3: GPU Implementation of Flocking Simulation

Original author: @hiroakioishi Translation and annotations by Claude

---

Introduction

This chapter explains how to implement flocking simulation using the Boids algorithm with ComputeShaders.

Birds, fish, and other animals sometimes form flocks or schools. The movement of these groups exhibits both regularity and complexity, possessing a certain beauty that has captivated people throughout history.

In computer graphics, controlling the behavior of each individual manually is impractical. To address this, an algorithm called **Boids** was devised for creating flocking behavior. This simulation algorithm consists of several simple rules and is easy to implement. However, a naive implementation requires checking the positional relationships with all other individuals, causing computational costs to increase with the square of the population size. When you want to control many individuals, CPU-based implementation becomes extremely difficult.

This is where we leverage the GPU's powerful parallel computing capabilities. Unity provides ComputeShaders for this kind of general-purpose GPU computing (GPGPU). GPUs have a special memory region called **shared memory**, and ComputeShaders allow us to effectively utilize this memory. Additionally, Unity has an advanced rendering feature called **GPU Instancing** that enables drawing large quantities of arbitrary meshes. This chapter introduces a program that uses these Unity GPU features to control and render many Boid objects.

> **Why This Matters**
>
> The Boids algorithm is a classic example of **emergent behavior**—complex, lifelike patterns arising from simple rules. It's used in: - Film and games (flocking birds, schools of fish, crowd simulation) - Screensavers and generative art - Robotics research (swarm intelligence)
>
> Understanding this chapter will teach you both the algorithm AND important GPU optimization techniques.

The Boids Algorithm

The Boids flocking simulation algorithm was developed by Craig Reynolds in 1986 and presented the following year at ACM SIGGRAPH 1987 in a paper titled "Flocks, Herds, and Schools: A Distributed Behavioral Model."

Reynolds observed that flocking behavior emerges when each individual modifies its own actions based on the positions and movement directions of surrounding individuals, perceived through senses like sight and hearing.

Each individual follows these **three simple behavioral rules**:

1. Separation

Move to avoid crowding with other individuals within a certain distance.

- **Think of it as:** “Don’t crash into your neighbors”

2. Alignment

Move toward the average heading direction of other individuals within a certain distance.

- **Think of it as:** “Fly in the same direction as your neighbors”

3. Cohesion

Move toward the average position of other individuals within a certain distance.

- **Think of it as:** “Stay close to your neighbors”

Boids Basic Rules *Figure: The three fundamental Boids rules*

By controlling individual movements according to these rules, we can program flocking behavior.

- **The Magic of Emergence**

  What’s remarkable is that NO individual knows about the “flock” as a whole. Each agent only follows local rules based on nearby neighbors. Yet from these simple rules, complex, organic-looking group behavior emerges. This is the essence of emergent systems.

Sample Program

Repository

https://github.com/IndieVisualLab/UnityGraphicsProgramming

Open the **BoidsSimulationOnGPU.unity** scene in the Assets/**BoidsSimulationOnGPU** folder of the sample Unity project.

Runtime Requirements

This program uses ComputeShaders and GPU Instancing.

ComputeShader works on the following platforms/APIs: - Windows and Windows Store apps with DirectX 11 or DirectX 12 graphics API and Shader Model 5.0 GPU - macOS and iOS using Metal graphics API - Android, Linux, and Windows platforms with Vulkan API - Modern OpenGL platforms (OpenGL 4.3 on Linux/Windows, OpenGL ES 3.1 on Android). Note: macOS does not support OpenGL 4.3 - Current-generation consoles (Sony PS4, Microsoft Xbox One)

GPU Instancing is available on: - DirectX 11 and DirectX 12 on Windows - OpenGL Core 4.1+/ES3.0+ on Windows, macOS, Linux, iOS, Android - Metal on macOS and iOS - Vulkan on Windows and Android - PlayStation 4 and Xbox One - WebGL (requires WebGL 2.0 API)

This sample uses the `Graphics.DrawMeshInstancedIndirect` method, requiring Unity 5.6 or later.

Implementation Code Explanation

The sample program consists of the following code:

File	Purpose
GPUBoids.cs	C# script that controls the ComputeShader for Boids simulation
Boids.compute	ComputeShader that performs the Boids simulation
BoidsRender.cs	C# script that controls the rendering shader
BoidsRender.shader	Shader for drawing objects via GPU instancing

Unity Editor Setup *Figure: Settings in the Unity Editor*

GPUBoids.cs - The Simulation Controller

This code manages the Boids simulation parameters, buffers needed for GPU computation, and the ComputeShader containing the calculation instructions.

Key Structure: BoidData

```
// Boid data structure
[System.Serializable]
struct BoidData
{
    public Vector3 Velocity; // Velocity
    public Vector3 Position; // Position
}
```

- **Why a struct?**

  GPU compute buffers work with contiguous memory blocks. By defining a struct, we ensure all boid data is laid out predictably in memory, making GPU access efficient.

Simulation Parameters

```
// Thread group thread size
const int SIMULATION_BLOCK_SIZE = 256;

// Maximum object count
[Range(256, 32768)]
public int MaxObjectNum = 16384;

// Radius for applying cohesion with other individuals
public float CohesionNeighborhoodRadius  = 2.0f;
// Radius for applying alignment with other individuals
public float AlignmentNeighborhoodRadius = 2.0f;
// Radius for applying separation with other individuals
public float SeparateNeighborhoodRadius  = 1.0f;

// Maximum velocity
public float MaxSpeed        = 5.0f;
// Maximum steering force
```

```
public float MaxSteerForce   = 0.5f;

// Weight for cohesion force
public float CohesionWeight  = 1.0f;
// Weight for alignment force
public float AlignmentWeight = 1.0f;
// Weight for separation force
public float SeparateWeight  = 3.0f;

// Weight for wall avoidance force
public float AvoidWallWeight = 10.0f;
```

Parameter Tuning

These weights dramatically affect behavior: - **High SeparateWeight**: Individuals spread out, avoid crowding - **High CohesionWeight**: Tight, dense flocks - **High AlignmentWeight**: Smooth, coordinated movement

Try adjusting these in the Unity Inspector to see different flocking behaviors!

ComputeBuffer Initialization

The `InitBuffer` function declares the buffers used for GPU computation.

```
void InitBuffer()
{
    // Initialize buffers
    _boidDataBuffer  = new ComputeBuffer(MaxObjectNum,
        Marshal.SizeOf(typeof(BoidData)));
    _boidForceBuffer = new ComputeBuffer(MaxObjectNum,
        Marshal.SizeOf(typeof(Vector3)));

    // Initialize Boid data and Force buffers
    var forceArr = new Vector3[MaxObjectNum];
    var boidDataArr = new BoidData[MaxObjectNum];
    for (var i = 0; i < MaxObjectNum; i++)
    {
        forceArr[i] = Vector3.zero;
        boidDataArr[i].Position = Random.insideUnitSphere * 1.0f;
        boidDataArr[i].Velocity = Random.insideUnitSphere * 0.1f;
    }
    _boidForceBuffer.SetData(forceArr);
    _boidDataBuffer.SetData(boidDataArr);
}
```

ComputeBuffer is a data buffer for storing data for ComputeShaders. It allows reading and writing to GPU memory buffers from C# scripts. The constructor arguments are the number of buffer elements and the size (in bytes) of each element. Using `Marshal.SizeOf()` retrieves the byte size of a type. With `SetData()`, you can set values from an array of any struct.

Executing ComputeShader Functions

The `Simulation` function passes necessary parameters to the ComputeShader and issues computation commands.

Functions in ComputeShaders that actually perform GPU calculations are called **kernels**. The execution unit for kernels is called a **thread**, and for parallel processing suited to GPU architecture, threads are

grouped together into **thread groups**.

The product of thread count and thread group count should equal or exceed the number of Boid objects.

```
void Simulation()
{
    ComputeShader cs = BoidsCS;
    int id = -1;

    // Calculate the number of thread groups
    int threadGroupSize = Mathf.CeilToInt(MaxObjectNum / SIMULATION_BLOCK_SIZE);

    // Calculate steering force
    id = cs.FindKernel("ForceCS"); // Get kernel ID
    cs.SetInt("_MaxBoidObjectNum", MaxObjectNum);
    cs.SetFloat("_CohesionNeighborhoodRadius", CohesionNeighborhoodRadius);
    // ... (set other parameters)
    cs.SetBuffer(id, "_BoidDataBufferRead", _boidDataBuffer);
    cs.SetBuffer(id, "_BoidForceBufferWrite", _boidForceBuffer);
    cs.Dispatch(id, threadGroupSize, 1, 1); // Execute ComputeShader

    // Calculate velocity and position from steering force
    id = cs.FindKernel("IntegrateCS"); // Get kernel ID
    cs.SetFloat("_DeltaTime", Time.deltaTime);
    cs.SetBuffer(id, "_BoidForceBufferRead", _boidForceBuffer);
    cs.SetBuffer(id, "_BoidDataBufferWrite", _boidDataBuffer);
    cs.Dispatch(id, threadGroupSize, 1, 1); // Execute ComputeShader
}
```

Kernels are specified using the `#pragma kernel` directive in the ComputeShader script. Each is assigned an ID, which can be retrieved from C# using `FindKernel`.

The `Dispatch` method issues a command to perform GPU computation on the kernel defined in the ComputeShader. Arguments include the kernel ID and the number of thread groups.

Boids.compute - The GPU Computation

This file contains the GPU computation instructions. There are two kernels: one calculates steering forces, and the other applies those forces to update velocity and position.

Buffer Declarations

```
// Kernel function specification
#pragma kernel ForceCS      // Calculate steering force
#pragma kernel IntegrateCS  // Calculate velocity, position

// Boid data structure
struct BoidData
{
    float3 velocity; // Velocity
    float3 position; // Position
};

// Thread group thread size
```

```
#define SIMULATION_BLOCK_SIZE 256

// Boid data buffer (read-only)
StructuredBuffer<BoidData>   _BoidDataBufferRead;
// Boid data buffer (read-write)
RWStructuredBuffer<BoidData> _BoidDataBufferWrite;
// Boid steering force buffer (read-only)
StructuredBuffer<float3>     _BoidForceBufferRead;
// Boid steering force buffer (read-write)
RWStructuredBuffer<float3>   _BoidForceBufferWrite;
```

- **Read vs Read-Write Buffers**

  - `StructuredBuffer<T>`: Read-only. Use when data won't change during the kernel.
  - `RWStructuredBuffer<T>`: Read-write. Use when you need to write results.

  This separation helps the GPU optimize memory access patterns.

Shared Memory - The Key Optimization

```
// Shared memory for Boid data
groupshared BoidData boid_data[SIMULATION_BLOCK_SIZE];
```

Variables with the `groupshared` storage modifier are written to **shared memory**. Shared memory can't hold large amounts of data, but it's placed close to registers and provides very fast access. This shared memory is accessible within a thread group.

By writing SIMULATION_BLOCK_SIZE worth of other individuals' data to shared memory at once and allowing fast reading within the same thread group, we can efficiently perform calculations that consider positional relationships with other individuals.

GPU Architecture *Figure: Basic GPU architecture*

- **Why Shared Memory Matters**

  Without this optimization, each thread would read other boids' data from slow global memory thousands of times. By loading a batch into fast shared memory and reusing it across the thread group, we get massive speedups.

  This is a fundamental GPU optimization pattern called **tiling**.

GroupMemoryBarrierWithGroupSync()

When accessing data written to shared memory, you must write `GroupMemoryBarrierWithGroupSync()` to synchronize all threads in the thread group.

This function blocks execution of all threads in the group until all threads have reached this call. This ensures that initialization of the `boid_data` array is properly completed across all threads in the thread group.

The Force Calculation Kernel

```
[numthreads(SIMULATION_BLOCK_SIZE, 1, 1)]
void ForceCS
(
    uint3 DTid : SV_DispatchThreadID, // Unique ID across all threads
    uint3 Gid : SV_GroupID,           // Group ID
    uint3 GTid : SV_GroupThreadID,    // Thread ID within group
    uint  GI : SV_GroupIndex          // SV_GroupThreadID as 1D (0-255)
```

```
)
{
    const unsigned int P_ID = DTid.x;
    float3 P_position = _BoidDataBufferRead[P_ID].position;
    float3 P_velocity = _BoidDataBufferRead[P_ID].velocity;

    float3 force = float3(0, 0, 0);

    // Variables for calculating the three rules
    float3 sepPosSum = float3(0, 0, 0); int sepCount = 0;
    float3 aliVelSum = float3(0, 0, 0); int aliCount = 0;
    float3 cohPosSum = float3(0, 0, 0); int cohCount = 0;

    // Process in blocks of SIMULATION_BLOCK_SIZE
    [loop]
    for (uint N_block_ID = 0; N_block_ID < (uint)_MaxBoidObjectNum;
        N_block_ID += SIMULATION_BLOCK_SIZE)
    {
        // Load SIMULATION_BLOCK_SIZE boids into shared memory
        boid_data[GI] = _BoidDataBufferRead[N_block_ID + GI];

        // Synchronize threads
        GroupMemoryBarrierWithGroupSync();

        // Calculate with other individuals
        for (int N_tile_ID = 0; N_tile_ID < SIMULATION_BLOCK_SIZE; N_tile_ID++)
        {
            float3 N_position = boid_data[N_tile_ID].position;
            float3 N_velocity = boid_data[N_tile_ID].velocity;

            float3 diff = P_position - N_position;
            float  dist = sqrt(dot(diff, diff));

            // --- Separation ---
            if (dist > 0.0 && dist <= _SeparateNeighborhoodRadius)
            {
                float3 repulse = normalize(P_position - N_position);
                repulse /= dist; // Closer = stronger repulsion
                sepPosSum += repulse;
                sepCount++;
            }

            // --- Alignment ---
            if (dist > 0.0 && dist <= _AlignmentNeighborhoodRadius)
            {
                aliVelSum += N_velocity;
                aliCount++;
            }

            // --- Cohesion ---
            if (dist > 0.0 && dist <= _CohesionNeighborhoodRadius)
            {
                cohPosSum += N_position;
                cohCount++;
```

```
            }
        }
        GroupMemoryBarrierWithGroupSync();
    }

    // Calculate steering forces from accumulated values
    // ... (steering force calculations)

    _BoidForceBufferWrite[P_ID] = force;
}
```

- **Understanding the Tiling Pattern**

  The outer loop processes boids in “tiles” of 256 at a time: 1. Load 256 boids from global memory into shared memory 2. Wait for all threads to finish loading 3. Each thread compares itself against all 256 loaded boids 4. Repeat for the next 256 boids

  This converts O(N) global memory reads into O(N/256) global reads + O(N) fast shared memory reads.

The Integration Kernel

```
[numthreads(SIMULATION_BLOCK_SIZE, 1, 1)]
void IntegrateCS(uint3 DTid : SV_DispatchThreadID)
{
    const unsigned int P_ID = DTid.x;

    BoidData b = _BoidDataBufferWrite[P_ID];
    float3 force = _BoidForceBufferRead[P_ID];

    // Apply repulsion when approaching walls
    force += avoidWall(b.position) * _AvoidWallWeight;

    b.velocity += force * _DeltaTime; // Apply force to velocity
    b.velocity = limit(b.velocity, _MaxSpeed); // Limit velocity
    b.position += b.velocity * _DeltaTime; // Update position

    _BoidDataBufferWrite[P_ID] = b;
}
```

GPU Instancing for Rendering

When you want to draw large quantities of identical meshes, creating individual GameObjects increases draw calls and rendering load. Additionally, transferring ComputeShader results to CPU memory is costly. For high-speed processing, we need to pass GPU computation results directly to the rendering shader.

Unity’s GPU Instancing allows drawing large quantities of identical meshes with minimal draw calls without creating unnecessary GameObjects.

Graphics.DrawMeshInstancedIndirect()

The **BoidsRender.cs** script uses `Graphics.DrawMeshInstancedIndirect` for GPU-instanced mesh rendering. This method allows passing mesh index counts and instance counts as ComputeBuffers, which is convenient when you want to read all instance data from the GPU.

```
void RenderInstancedMesh()
{
    // Get index count of specified mesh
    uint numIndices = (InstanceMesh != null) ?
        (uint)InstanceMesh.GetIndexCount(0) : 0;
    args[0] = numIndices;
    args[1] = (uint)GPUBoidsScript.GetMaxObjectNum();
    argsBuffer.SetData(args);

    // Set Boid data buffer to material
    InstanceRenderMaterial.SetBuffer("_BoidDataBuffer",
        GPUBoidsScript.GetBoidDataBuffer());
    InstanceRenderMaterial.SetVector("_ObjectScale", ObjectScale);

    // Define bounds
    var bounds = new Bounds(
        GPUBoidsScript.GetSimulationAreaCenter(),
        GPUBoidsScript.GetSimulationAreaSize()
    );

    // Draw mesh with GPU instancing
    Graphics.DrawMeshInstancedIndirect(
        InstanceMesh,           // Mesh to instance
        0,                      // Submesh index
        InstanceRenderMaterial, // Rendering material
        bounds,                 // Bounds
        argsBuffer              // Arguments buffer
    );
}
```

The Power of Indirect Drawing

`DrawMeshInstancedIndirect` is special because the instance count comes from a GPU buffer, not a CPU variable. This means: - No CPU-GPU synchronization needed - Instance count can be determined by GPU computation - Perfect for particle systems, procedural content, etc.

The Rendering Shader

The **BoidsRender.shader** is designed to work with `Graphics.DrawMeshInstancedIndirect`.

Key Shader Setup

```
#pragma surface surf Standard vertex:vert addshadow
#pragma instancing_options procedural:setup
```

The `procedural:setup` directive tells Unity to generate additional variants for use with `Graphics.DrawMeshInstancedIndirect`. The specified function (`setup`) is called at the beginning of the vertex shader stage.

Vertex Shader - Transforming Each Boid

```
void vert(inout appdata_full v)
{
```

```
    #ifdef UNITY_PROCEDURAL_INSTANCING_ENABLED
    // Get Boid data from instance ID
    BoidData boidData = _BoidDataBuffer[unity_InstanceID];

    float3 pos = boidData.position.xyz;
    float3 scl = _ObjectScale;

    // Create object-to-world transformation matrix
    float4x4 object2world = (float4x4)0;
    object2world._11_22_33_44 = float4(scl.xyz, 1.0);

    // Calculate rotation from velocity
    float rotY = atan2(boidData.velocity.x, boidData.velocity.z);
    float rotX = -asin(boidData.velocity.y /
        (length(boidData.velocity.xyz) + 1e-8));

    float4x4 rotMatrix = eulerAnglesToRotationMatrix(float3(rotX, rotY, 0));
    object2world = mul(rotMatrix, object2world);
    object2world._14_24_34 += pos.xyz;

    // Transform vertex and normal
    v.vertex = mul(object2world, v.vertex);
    v.normal = normalize(mul(object2world, v.normal));
    #endif
}
```

Using `unity_InstanceID`, you can get a unique ID for each instance. By using this ID as an index into the StructuredBuffer declared as Boid data buffer, you can obtain unique Boid data for each instance.

Calculating Rotation from Velocity

The rotation is calculated from the Boid's velocity data so it faces its direction of travel. Using Euler angles for intuitive handling:

- **Yaw** (horizontal direction): Calculated using `atan2` from Z and X velocity components
- **Pitch** (vertical tilt): Calculated using `asin` from the ratio of Y velocity to total velocity magnitude

Roll Pitch Yaw *Figure: Axis rotation terminology*

Results

With this implementation, you should see objects that move with flock-like behavior.

Execution Result *Figure: Execution result*

Summary

The implementation introduced in this chapter uses the minimum Boids algorithm, but through parameter adjustment, flocks can form large groups or split into many smaller groups, showing different movement characteristics.

Beyond the basic behavioral rules shown here, other rules should be considered. For example, if these were fish and a predator appeared, they would naturally flee. If there were terrain or obstacles, the fish

would avoid collisions. Regarding vision, different animal species have different fields of view and acuity—excluding individuals outside the field of view from calculations would make the simulation more realistic.

Whether flying through air, swimming in water, or moving on land, the environment and characteristics of locomotor organs also affect movement patterns. Individual differences should also be considered.

Performance Considerations

GPU parallel processing can calculate many more individuals than CPU-based computation, but fundamentally, calculations with other individuals are done exhaustively ($O(N^2)$), which isn't very efficient. To reduce computational cost, you could: - Register individuals in grid or block regions based on their positions - Only calculate with individuals in adjacent regions (spatial hashing)

This **neighbor search optimization** can significantly reduce computational costs for very large populations.

Next Steps for Exploration

- Try different mesh shapes (fish, birds, abstract shapes)
- Add predator/prey relationships
- Implement obstacle avoidance
- Experiment with spatial partitioning for better performance
- Add visual trails or other effects

References

- Boids Background and Update - https://www.red3d.com/cwr/boids/
- THE NATURE OF CODE - http://natureofcode.com/
- Real-Time Particle Systems on the GPU in Dynamic Environments
- Practical Rendering and Computation with Direct3D 11
- GPU Parallel Graphics Processing Introduction (Japanese)

Key Takeaways

What You Should Remember

1. **Boids = 3 simple rules** Separation, Alignment, Cohesion
2. **GPU optimization with shared memory** Load data in tiles, reuse across thread group
3. **GroupMemoryBarrierWithGroupSync()** Essential for shared memory coordination
4. **GPU Instancing** Draw thousands of meshes with minimal draw calls
5. **DrawMeshInstancedIndirect** Instance count from GPU buffer, no CPU sync needed
6. **Vertex shader transformation** Use `unity_InstanceID` to get per-instance data

---

Next chapter: Grid-based Fluid Simulation with Stable Fluids!

Chapter 4: Grid-Based Fluid Simulation

Original author: @sakota Translation and annotations by Claude

---

About This Chapter

This chapter explains grid-based fluid simulation using ComputeShaders, based on the famous "Stable Fluids" paper by Jos Stam.

Sample Data

Code

https://github.com/IndieVisualLab/UnityGraphicsProgramming

Located in the Assets/**StableFluids** folder.

Runtime Requirements

- Environment supporting ComputeShaders with Shader Model 5.0
- Tested with Unity 5.6.2 and Unity 2017.1.1

Introduction

This chapter explains grid-based fluid simulation and the mathematical concepts needed to implement it. But first, what exactly is "grid-based" simulation? To understand this, let's briefly explore how fluid mechanics analyzes "flow."

Approaches in Fluid Mechanics

Fluid mechanics is characterized by mathematically formulating natural phenomena of "flow" to make them computationally tractable. But how can we quantify and analyze this "flow"?

Put simply, we can quantify it by deriving "the velocity at a moment in time." Mathematically speaking, this means analyzing the change in velocity vectors when taking time derivatives.

However, there are **two fundamental approaches** to analyzing flow:

Approach 1: Eulerian Method Imagine water in a bathtub. Divide the water into a grid of fixed cells, and measure the velocity vector at each fixed grid location.

Approach 2: Lagrangian Method Imagine dropping a rubber duck in the bathtub. Track and analyze the duck's movement itself.

> **Why This Matters**
>
> These two perspectives—fixed observation points (Eulerian) vs. moving with the flow (Lagrangian)—are fundamental to all fluid simulation. Understanding this distinction helps you choose the right approach for different effects.

Types of Fluid Simulation

In computer graphics, fluid simulations can be broadly categorized into three types:

Type	Example	Description
Grid-based	Stable Fluids	Like the Eulerian approach—simulate velocity at fixed grid points

Type	Example	Description
Particle-based	SPH (Smoothed Particle Hydrodynamics)	Like the Lagrangian approach—track individual particles
Hybrid	FLIP, PIC	Combines both approaches

As you might guess from these names: - **Grid-based methods** create a "field" of grid cells and simulate velocity at each cell (Eulerian style) - **Particle-based methods** focus on individual particles and simulate their movement (Lagrangian style)

Each approach has strengths and weaknesses:

Aspect	Grid-Based	Particle-Based
Pressure calculation	Good	Challenging
Viscosity/Diffusion	Good	Challenging
Advection (transport)	Challenging	Good

> **Why the Difference?**
>
> Grid methods work well for differential operators (pressure, diffusion) because neighboring cells are at known, fixed positions. But advection moves things between cells, which creates interpolation challenges.
>
> Particle methods naturally handle advection (particles just move!) but struggle with pressure because finding neighboring particles requires expensive spatial searches.

Hybrid methods like FLIP were developed to get the best of both worlds.

This chapter focuses on **Stable Fluids**, the seminal paper by Jos Stam presented at SIGGRAPH 1999, which describes grid-based simulation of incompressible viscous fluids.

The Navier-Stokes Equations

Let's examine the Navier-Stokes equations for grid-based simulation:

The Velocity Field Equation

$$\frac{\partial \vec{u}}{\partial t} = -(\vec{u} \cdot \nabla)\vec{u} + \nu \nabla^2 \vec{u} + \vec{f}$$

The Density Field Equation

$$\frac{\partial \rho}{\partial t} = -(\vec{u} \cdot \nabla)\rho + \kappa \nabla^2 \rho + S$$

The Continuity Equation (Mass Conservation)

$$\nabla \cdot \vec{u} = 0$$

The first equation describes the **velocity field**, the second describes the **density field**, and the third is the **continuity equation** (mass conservation law).

Let's unpack each of these.

Don't Panic!

These equations look intimidating, but we'll break them down piece by piece. Each term has a clear physical meaning, and the implementation follows naturally from understanding these meanings.

The Continuity Equation (Mass Conservation)

Let's start with the simplest equation—the continuity equation—which serves as a constraint for simulating **incompressible** fluids.

When simulating fluids, we must clearly distinguish whether the target is **compressible** or **incompressible**:

- **Compressible fluids**: Density changes with pressure (e.g., gases)
- **Incompressible fluids**: Density is constant everywhere (e.g., water)

This chapter deals with incompressible fluids, so we must keep the **divergence** of the velocity field at zero in each cell. This means inflow and outflow must cancel out—if there's inflow, there must be equal outflow, causing velocity to propagate. This condition is expressed as:

$$\nabla \cdot \vec{u} = 0$$

This means "divergence equals zero." But what exactly is divergence?

Divergence

$$\nabla \cdot \vec{u} = \nabla \cdot (u, v) = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$$

The **nabla operator** (∇) is a vector differential operator. For a 2D vector field, it represents the partial derivative notation $\left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}\right)$.

The nabla operator itself has no meaning alone—its operation changes depending on whether it's combined with a dot product (divergence), cross product (curl), or simply applied to a function (gradient).

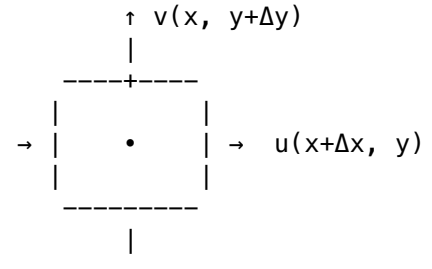
Intuition: What Does Divergence Mean?

Divergence measures how much a vector field "spreads out" or "converges" at a point: - **Positive divergence**: Source (things spreading outward) - **Negative divergence**: Sink (things converging inward) - **Zero divergence**: What flows in must flow out

For incompressible fluids, we enforce zero divergence everywhere—no sources or sinks allowed!

Understanding Divergence Geometrically

To understand why the formula works, consider a single cell from the grid:



`↓ v(x, y)`

Divergence calculates the net "outflow" from this cell. We can derive it by considering the flow across each boundary:

$$\frac{i(x+\Delta x,y)\Delta y - i(x,y)\Delta y + j(x,y+\Delta y)\Delta x - j(x,y)\Delta x}{\Delta x \Delta y}$$

$$= \frac{i(x+\Delta x,y) - i(x,y)}{\Delta x} + \frac{j(x,y+\Delta y) - j(x,y)}{\Delta y}$$

Taking the limit as Δ approaches zero:

$$\lim_{\Delta x \to 0} \frac{i(x+\Delta x,y) - i(x,y)}{\Delta x} + \lim_{\Delta y \to 0} \frac{j(x,y+\Delta y) - j(x,y)}{\Delta y} = \frac{\partial i}{\partial x} + \frac{\partial j}{\partial y}$$

This shows that divergence equals the sum of partial derivatives—exactly our formula!

The Velocity Field

Now let's tackle the main event: the velocity field equation.

Before implementing it, we need to understand two more differential operators in addition to divergence: **gradient** and **Laplacian**.

Gradient

$$\nabla f(x,y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)$$

The gradient (∇f or grad f) computes the direction of steepest increase. By sampling a function at infinitesimally displaced points in each direction and combining the results, we get a vector pointing "uphill."

> **Intuition: What Does Gradient Mean?**
>
> Imagine standing on a hillside. The gradient points in the direction of steepest ascent, and its magnitude tells you how steep the slope is. In fluid simulation, we use gradients to find where pressure differences will push the fluid.

Laplacian

$$\Delta f = \nabla^2 f = \nabla \cdot \nabla f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

The Laplacian (written as $\nabla^2 f$ or $\nabla \cdot \nabla f$) is a second-order differential operator. Think of it as taking the gradient (direction of increase), then computing the divergence of that result.

> **Intuition: What Does Laplacian Mean?**
>
> The Laplacian measures how different a value is from its neighbors' average: - **Positive Laplacian**: Value is less than neighbors' average (local minimum) - **Negative Laplacian**: Value is greater than neighbors' average (local maximum) - **Zero Laplacian**: Value equals neighbors' average

This is why the Laplacian naturally describes diffusion—values tend to spread toward equilibrium with their neighbors.

For vector fields, in Cartesian coordinates, we can compute the Laplacian component-wise:

$$\nabla^2 \vec{u} = \left(\frac{\partial^2 u_x}{\partial x^2} + \frac{\partial^2 u_x}{\partial y^2} + \frac{\partial^2 u_x}{\partial z^2}, \frac{\partial^2 u_y}{\partial x^2} + \frac{\partial^2 u_y}{\partial y^2} + \frac{\partial^2 u_y}{\partial z^2}, \frac{\partial^2 u_z}{\partial x^2} + \frac{\partial^2 u_z}{\partial y^2} + \frac{\partial^2 u_z}{\partial z^2} \right)$$

Now we have all the mathematical tools needed to understand the Navier-Stokes equation!

Breaking Down the Velocity Field Equation

$$\frac{\partial \vec{u}}{\partial t} = -(\vec{u} \cdot \nabla)\vec{u} + \nu \nabla^2 \vec{u} + \vec{f}$$

Where: - $\vec{u}$ = velocity - ν = kinematic viscosity coefficient - $\vec{f}$ = external force

The **left side** represents velocity change over time. The **right side** has three terms:

Term	Name	Physical Meaning
$-(\vec{u} \cdot \nabla)\vec{u}$	Advection	Velocity carrying itself along
$\nu \nabla^2 \vec{u}$	Diffusion/Viscosity	Velocity spreading to neighbors
$\vec{f}$	External Force	Applied forces (user input, gravity, etc.)

> **Key Implementation Insight**
>
> Although mathematically these terms combine in one equation, we implement them as **separate steps** executed sequentially. This is called **operator splitting** and is what makes Stable Fluids "stable"—each step can be solved robustly.

Let's implement each term, starting with external forces (since nothing happens without an initial push!).

Velocity Field: External Force Term

This is the simplest part—just add external vectors to the velocity field. This could be from user input (mouse drag), gravity, or other events.

```
float visc;          // Kinematic viscosity coefficient
float dt;            // Delta time
float velocityCoef;  // Velocity force coefficient
float densityCoef;   // Density force coefficient

// xy = velocity, z = density - the fluid solver output
RWTexture2D<float4> solver;
// Density field
RWTexture2D<float>  density;
// Velocity field
RWTexture2D<float2> velocity;
// xy = previous velocity, z = previous density
// Also used for temporary storage during projection
```

```
RWTexture2D<float3> prev;
// xy = velocity source, z = density source - external input buffer
Texture2D source;

[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void AddSourceVelocity(uint2 id : SV_DispatchThreadID)
{
    uint w, h;
    velocity.GetDimensions(w, h);

    if (id.x < w && id.y < h)
    {
        velocity[id] += source[id].xy * velocityCoef * dt;
        prev[id] = float3(source[id].xy * velocityCoef * dt, prev[id].z);
    }
}
```

Velocity Field: Diffusion/Viscosity Term

$$\nu \nabla^2 \vec{u}$$

The nabla operator acts only on what's to its right, so first we compute the vector Laplacian, then multiply by the viscosity coefficient.

The Laplacian takes the gradient and divergence of each velocity component, causing the velocity to **diffuse** (spread) to neighboring cells. The viscosity coefficient controls how fast this spreading occurs.

The Stability Problem

If we implement this directly (explicit method), high diffusion rates can cause **oscillation** and eventually **numerical explosion**—the simulation becomes unstable and diverges.

The Solution: Iterative Methods

To achieve stable diffusion, we use iterative methods like **Gauss-Seidel**, **Jacobi**, or **SOR**. These convert the equation into a form that converges to the correct answer through repeated iteration.

> **Why Gauss-Seidel Works**
>
> Instead of computing the new value directly, Gauss-Seidel solves: "What value would diffuse to produce my current state?" This implicit formulation is unconditionally stable—it won't explode regardless of time step or viscosity!

More iterations = more accurate results, but in real-time graphics, we prioritize framerate and visual quality over physical accuracy. Tune the iteration count based on performance and appearance.

```
#define GS_ITERATE 2  // Gauss-Seidel iterations. Directly affects performance.

[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void DiffuseVelocity(uint2 id : SV_DispatchThreadID)
{
    uint w, h;
    velocity.GetDimensions(w, h);

    if (id.x < w && id.y < h)
```

```
    {
        float a = dt * visc * w * h;

        [unroll]
        for (int k = 0; k < GS_ITERATE; k++) {
            velocity[id] = (prev[id].xy + a * (
                            velocity[int2(id.x - 1, id.y)] +
                            velocity[int2(id.x + 1, id.y)] +
                            velocity[int2(id.x, id.y - 1)] +
                            velocity[int2(id.x, id.y + 1)]
                            )) / (1 + 4 * a);
            SetBoundaryVelocity(id, w, h);
        }
    }
}
```

Understanding the Formula

The formula `(prev + a * neighbors_sum) / (1 + 4*a)` is the Gauss-Seidel solution to the diffusion equation. The a term represents `dt * viscosity * gridSize`, and we're solving for the velocity that, when diffused, produces the current state.

The `SetBoundaryVelocity` function handles boundary conditions. See the repository for details.

Mass Conservation (Projection)

$$\nabla \cdot \vec{u} = 0$$

Before proceeding further, let's return to mass conservation. After external forces and diffusion, each cell has accumulated velocity—but mass isn't conserved yet. Some cells have net outflow (sources) while others have net inflow (sinks).

We must enforce the constraint that divergence equals zero everywhere. This step, called **projection**, makes the fluid behave realistically—outflow from one area causes inflow elsewhere, propagating the flow.

Implementation Challenge

When computing partial derivatives on the GPU, we need neighboring cells' values to be "finalized." Using group shared memory would be faster, but cross-group access causes artifacts. So we split projection into **three separate kernel dispatches**:

1. **Step 1**: Compute divergence from velocity field
2. **Step 2**: Solve Poisson equation using Gauss-Seidel
3. **Step 3**: Subtract gradient to make divergence zero

```
// Mass Conservation Step 1: Compute divergence from velocity field
[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void ProjectStep1(uint2 id : SV_DispatchThreadID)
{
    uint w, h;
    velocity.GetDimensions(w, h);

    if (id.x < w && id.y < h)
    {
        float2 uvd;
```

```
        uvd = float2(1.0 / w, 1.0 / h);

        // Store divergence in prev.y, pressure in prev.x
        prev[id] = float3(0.0,
                    -0.5 *
                    (uvd.x * (velocity[int2(id.x + 1, id.y)].x -
                              velocity[int2(id.x - 1, id.y)].x)) +
                    (uvd.y * (velocity[int2(id.x, id.y + 1)].y -
                              velocity[int2(id.x, id.y - 1)].y)),
                    prev[id].z);

        SetBoundaryDivergence(id, w, h);
        SetBoundaryDivPositive(id, w, h);
    }
}

// Mass Conservation Step 2: Solve Poisson equation via Gauss-Seidel
[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void ProjectStep2(uint2 id : SV_DispatchThreadID)
{
    uint w, h;

    velocity.GetDimensions(w, h);

    if (id.x < w && id.y < h)
    {
        for (int k = 0; k < GS_ITERATE; k++)
        {
            prev[id] = float3(
                        (prev[id].y + prev[uint2(id.x - 1, id.y)].x +
                                      prev[uint2(id.x + 1, id.y)].x +
                                      prev[uint2(id.x, id.y - 1)].x +
                                      prev[uint2(id.x, id.y + 1)].x) / 4,
                        prev[id].yz);
            SetBoundaryDivPositive(id, w, h);
        }
    }
}

// Mass Conservation Step 3: Make divergence zero
[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void ProjectStep3(uint2 id : SV_DispatchThreadID)
{
    uint w, h;

    velocity.GetDimensions(w, h);

    if (id.x < w && id.y < h)
    {
        float  velX, velY;
        float2 uvd;
        uvd = float2(1.0 / w, 1.0 / h);

        velX = velocity[id].x;
```

```
        velY = velocity[id].y;

        // Subtract pressure gradient
        velX -= 0.5 * (prev[uint2(id.x + 1, id.y)].x -
                       prev[uint2(id.x - 1, id.y)].x) / uvd.x;
        velY -= 0.5 * (prev[uint2(id.x, id.y + 1)].x -
                       prev[uint2(id.x, id.y - 1)].x) / uvd.y;

        velocity[id] = float2(velX, velY);
        SetBoundaryVelocity(id, w, h);
    }
}
```

The Pressure-Velocity Connection

This projection step finds a "pressure" field that, when we subtract its gradient from velocity, gives us a divergence-free field. Physically, this represents how pressure differences push fluid to maintain incompressibility.

Now the velocity field conserves mass—where there's outflow, inflow occurs to compensate, creating realistic fluid propagation!

Velocity Field: Advection Term

$$-(\vec{u} \cdot \nabla)\vec{u}$$

Advection is where the Lagrangian perspective enters our Eulerian grid method. We need to move velocity values along with the flow.

The Backtrace Method

For each grid cell, we trace **backward** along the velocity to find where the current value "came from" in the previous timestep. We then copy that value to the current cell.

Since the backtrace point rarely lands exactly on a grid cell, we use **bilinear interpolation** with the four nearest cells to get the correct value.

```
[numthreads(THREAD_X, THREAD_Y, THREAD_Z)]
void AdvectVelocity(uint2 id : SV_DispatchThreadID)
{
    uint w, h;
    density.GetDimensions(w, h);

    if (id.x < w && id.y < h)
    {
        int ddx0, ddx1, ddy0, ddy1;
        float x, y, s0, t0, s1, t1, dfdt;

        dfdt = dt * (w + h) * 0.5;

        // Calculate backtrace point
        x = (float)id.x - dfdt * prev[id].x;
        y = (float)id.y - dfdt * prev[id].y;

        // Clamp to simulation bounds
```

```
        clamp(x, 0.5, w + 0.5);
        clamp(y, 0.5, h + 0.5);

        // Find neighboring cells for interpolation
        ddx0 = floor(x);
        ddx1 = ddx0 + 1;
        ddy0 = floor(y);
        ddy1 = ddy0 + 1;

        // Calculate interpolation weights
        s1 = x - ddx0;
        s0 = 1.0 - s1;
        t1 = y - ddy0;
        t0 = 1.0 - t1;

        // Bilinear interpolation from backtraced position
        velocity[id] = s0 * (t0 * prev[int2(ddx0, ddy0)].xy +
                             t1 * prev[int2(ddx0, ddy1)].xy) +
                       s1 * (t0 * prev[int2(ddx1, ddy0)].xy +
                             t1 * prev[int2(ddx1, ddy1)].xy);
        SetBoundaryVelocity(id, w, h);
    }
}
```

Why Backtrace Instead of Forward?

Forward advection (pushing values to where they'll go) causes gaps and overlaps—some cells receive multiple values, others none. Backtracing (pulling from where values came) guarantees every cell gets exactly one value. This is a key insight of Stable Fluids!

The Density Field

Now let's look at the density field equation:

$$\frac{\partial \rho}{\partial t} = -(\vec{u} \cdot \nabla)\rho + \kappa \nabla^2 \rho + S$$

Where: - $\vec{u}$ = velocity - κ = diffusion coefficient - ρ = density - S = external density source

The density field isn't strictly necessary, but it allows us to create visual effects like smoke or ink by having pixels "ride" on the velocity field, diffusing and flowing together.

Notice that this equation has the **same structure** as the velocity field equation: - Advection term: $-(\vec{u}\cdot\nabla)\rho$ - Diffusion term: $\kappa\nabla^2\rho$ - Source term: S

The only differences are: 1. Vectors become scalars (density is just a number) 2. Kinematic viscosity (ν) becomes diffusion coefficient (κ) 3. **No mass conservation required** (density can compress/expand freely)

Since density represents concentration, not physical mass in our simulation, we don't need the divergence-free constraint. This makes the density step simpler than the velocity step.

The implementation reuses the same kernels with reduced dimensionality. See the repository for the density field implementation.

Simulation Step Order

Here's the complete simulation loop order:

Each Frame:

1. **Apply external forces** to velocity and density fields
2. **Update velocity field:**
   - Diffusion (viscosity)
   - **Mass conservation (projection)**
   - Advection
   - **Mass conservation (projection)**
3. **Update density field:**
   - Diffusion
   - Advection (using velocity field)

Why Project Twice?

We apply mass conservation after both diffusion AND advection because each step can introduce divergence. The second projection ensures the final velocity field is truly divergence-free.

C# Controller: Dispatching the Kernels

The C# code orchestrates these kernel dispatches in the correct order:

```
protected override void VelocityStep()
{
    // Add velocity source to velocity field
    if (SorceTex != null)
    {
        computeShader.SetTexture(kernelMap[ComputeKernels.AddSourceVelocity], sourceId, SorceTe
        computeShader.SetTexture(kernelMap[ComputeKernels.AddSourceVelocity], velocityId, veloc
        computeShader.SetTexture(kernelMap[ComputeKernels.AddSourceVelocity], prevId, prevTex);
        computeShader.Dispatch(kernelMap[ComputeKernels.AddSourceVelocity],
            Mathf.CeilToInt(velocityTex.width / gpuThreads.x),
            Mathf.CeilToInt(velocityTex.height / gpuThreads.y), 1);
    }

    // Diffuse velocity
    // ... dispatch DiffuseVelocity

    // Project (3 steps)
    // ... dispatch ProjectStep1, ProjectStep2, ProjectStep3

    // Swap buffers
    // ... dispatch SwapVelocity

    // Advect velocity
    // ... dispatch AdvectVelocity

    // Project again (3 steps)
```

```
    // ... dispatch ProjectStep1, ProjectStep2, ProjectStep3
}
```

Results

Running the simulation and dragging the mouse on screen produces fluid simulation like this:

Execution Result *Figure: Execution result*

Summary

Fluid simulation is computationally expensive—unlike pre-rendered effects, real-time game engines face significant performance constraints. However, with improving GPU capabilities, 2D simulations at reasonable resolutions can now achieve acceptable framerates.

Performance Optimization Ideas

- Replace Gauss-Seidel iterations with faster alternatives
- Substitute the velocity field with curl noise for lighter computation
- Reduce grid resolution and upscale results
- Skip projection steps for artistic (non-physical) effects

Going Further

If this chapter sparked your interest in fluids, definitely try the next chapter on **Particle-Based Fluid Simulation (SPH)**. It approaches fluid from a completely different angle, giving you a deeper appreciation for the richness and implementation challenges of fluid simulation.

References

- Jos Stam, SIGGRAPH 1999, “Stable Fluids”

Key Takeaways

What You Should Remember

1. **Eulerian vs Lagrangian**: Fixed grid vs. moving particles—fundamental fluid perspectives
2. **Navier-Stokes terms**: Force Diffusion Projection Advection Projection
3. **Divergence = 0**: Incompressible fluids have no sources or sinks
4. **Gauss-Seidel**: Iterative solver for stable diffusion and projection
5. **Backtracing**: Pull values from where they came (not push where they’re going)
6. **Operator splitting**: Solve complex equations by tackling one term at a time
7. **Project twice**: After diffusion AND after advection to maintain divergence-free velocity

---

Mathematical Operator Quick Reference

Operator	Symbol	Formula (2D)	Meaning
Gradient	∇f	$(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y})$	Direction of steepest increase
Divergence	$\nabla \cdot \vec{u}$	$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$	Net outflow from a point
Laplacian	$\nabla^2 f$	$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$	Difference from neighbors' average

---

Next chapter: SPH Particle-Based Fluid Simulation!

Chapter 5: SPH Particle-Based Fluid Simulation

Original author: @takao Translation and annotations by Claude

---

Introduction

The previous chapter covered grid-based fluid simulation. This chapter explores the other major approach to fluid simulation: **particle methods**, specifically **SPH (Smoothed Particle Hydrodynamics)**.

Some explanations are simplified for clarity—please understand if certain expressions are incomplete.

Background Knowledge

Eulerian vs Lagrangian Perspectives

There are two fundamental ways to observe fluid motion:

Eulerian Perspective: Fix observation points in space at regular intervals and analyze fluid motion at those fixed locations.

Lagrangian Perspective: Float observation points that move with the flow and analyze fluid motion from those moving viewpoints.

Generally: - **Grid-based methods** = Eulerian perspective - **Particle methods** = Lagrangian perspective

> **Why This Matters**
>
> The Eulerian view is like standing on a bridge watching water flow past. The Lagrangian view is like riding a leaf floating downstream. Each perspective leads to different mathematical formulations and implementation approaches.

The Lagrangian Derivative (Material Derivative)

The mathematical treatment differs between these perspectives. In the Eulerian view, a physical quantity (like velocity or density) at position $\vec{x}$ and time t is written as:

$$\phi = \phi(\vec{x}, t)$$

Its time derivative is simply:

$$\frac{\partial \phi}{\partial t}$$

This represents the rate of change at a **fixed** position—the Eulerian derivative.

In the Lagrangian view, observation points move with the flow (**advection**). A point initially at $\vec{x}_0$ is at time t located at:

$$\vec{x}(\vec{x}_0, t)$$

So the physical quantity becomes:

$$\phi = \phi(\vec{x}(\vec{x}_0, t), t)$$

Following the definition of derivatives and using the chain rule:

$$\lim_{\Delta t \to 0} \frac{\phi(\vec{x}(\vec{x}_0, t + \Delta t), t + \Delta t) - \phi(\vec{x}(\vec{x}_0, t), t)}{\Delta t}$$

$$= \sum_i \frac{\partial \phi}{\partial x_i} \frac{\partial x_i}{\partial t} + \frac{\partial \phi}{\partial t}$$

$$= \left(\frac{\partial}{\partial t} + \vec{u} \cdot \text{grad} \right) \phi$$

To simplify notation, we introduce the **material derivative operator**:

$$\frac{D}{Dt} := \frac{\partial}{\partial t} + \vec{u} \cdot \text{grad}$$

> **Key Insight**
>
> The material derivative has two parts: - $\frac{\partial}{\partial t}$: Change at a fixed point (local change) - $\vec{u} \cdot$ grad: Change due to moving to a new location (convective change)
>
> In particle methods, since particles naturally move with the flow, the material derivative becomes much simpler to compute!

The Incompressibility Condition

When fluid velocity is much slower than the speed of sound, we can assume no volume change occurs. This **incompressibility condition** is expressed as:

$$\nabla \cdot \vec{u} = 0$$

This means there are no sources or sinks within the fluid—what flows in must flow out.

Particle Method Simulation

Particle methods divide fluid into small **particles** and observe motion from the Lagrangian perspective. These particles are our moving observation points.

Many particle methods exist:

- **SPH** (Smoothed Particle Hydrodynamics)
- **FLIP** (Fluid Implicit Particle)
- **PIC** (Particle In Cell)
- **MPS** (Moving Particle Semi-implicit)
- **MPM** (Material Point Method)

Deriving Navier-Stokes for Particle Methods

The particle-method form of the Navier-Stokes equation is:

$$\frac{D\vec{u}}{Dt} = -\frac{1}{\rho}\nabla p + \nu\nabla \cdot \nabla\vec{u} + \vec{g}$$

Notice something different from the grid-based version? The **advection term is gone**!

> **Why No Advection Term?**
>
> Remember the relationship between Eulerian and Lagrangian derivatives. In particle methods, observation points move with the flow, so advection is handled automatically by moving the particles. We don't need to calculate it separately in the NS equation!

Real fluids are collections of molecules—essentially a particle system. But simulating actual molecules is computationally impossible, so we use larger "blobs" representing fluid regions.

Each particle has: - Mass m - Position $\vec{x}$ - Velocity $\vec{u}$ - Volume V

For each particle, we calculate external forces $\vec{f}$, solve Newton's equation $m\vec{a} = \vec{f}$ to get acceleration, and determine movement for the next timestep.

Force 1: Pressure

Fluid always flows from high pressure to low pressure regions. Taking the gradient of the pressure scalar field gives us the direction of maximum pressure increase. Since force acts from high to low pressure, we negate it: $-\nabla p$.

Since particles have volume, the pressure force on a particle is $-V\nabla p$.

Force 2: Viscosity

Viscous fluids (honey, melted chocolate) resist deformation. In particle terms: **a particle's velocity tends toward the average of its neighbors' velocities**.

The Laplacian operator computes this neighborhood averaging, giving us: $\mu\nabla \cdot \nabla\vec{u}$

where μ is the **dynamic viscosity coefficient**.

Combining Forces

Putting pressure, viscosity, and external forces (gravity) into Newton's equation:

$$m\frac{D\vec{u}}{Dt} = -V\nabla p + V\mu\nabla\cdot\nabla\vec{u} + m\vec{g}$$

Since $m = \rho V$:

$$\rho\frac{D\vec{u}}{Dt} = -\nabla p + \mu\nabla\cdot\nabla\vec{u} + \rho\vec{g}$$

Dividing by ρ and substituting $\nu = \frac{\mu}{\rho}$ (kinematic viscosity):

$$\frac{D\vec{u}}{Dt} = -\frac{1}{\rho}\nabla p + \nu\nabla\cdot\nabla\vec{u} + \vec{g}$$

This is our particle-method Navier-Stokes equation!

Advection in Particle Methods

Since particles ARE the observation points, advection is simply moving the particles. Using finite differences with small timestep Δt:

Velocity update:

$$\vec{u}_{n+1} = \vec{u}_n + \Delta t\cdot\vec{a}$$

Position update:

$$\vec{x}_{n+1} = \vec{x}_n + \Delta t\cdot\vec{u}$$

This is called the **Forward Euler method**.

SPH Fluid Simulation

We can't solve differential equations directly on computers—we need approximations. **SPH (Smoothed Particle Hydrodynamics)** provides this approximation.

SPH originated in astrophysics for simulating celestial body collisions. Desbrun et al. (1996) adapted it for CG fluid simulation. It parallelizes well and modern GPUs can handle massive particle counts in real-time.

The Kernel Function (Weight Function)

In SPH, each particle has an **influence radius**. Nearby particles have stronger influence; distant particles have less.

This influence is described by a **kernel function** (or weight function) W. The kernel is shaped like a smooth bump centered on the particle, falling to zero at the influence radius h.

> **Intuition**
>
> Think of each particle as a soft blob of influence. When calculating properties at any point, you sum contributions from all nearby particles, weighted by how close they are. Closer = more influence.

Discretizing Physical Quantities

Any physical quantity ϕ at position $\vec{x}$ can be approximated as:

$$\phi(\vec{x}) = \sum_{j \in N} m_j \frac{\phi_j}{\rho_j} W(\vec{x}_j - \vec{x}, h)$$

Where: - N = set of neighboring particles - m = particle mass - ρ = particle density - h = influence radius - W = kernel function

Gradient:

$$\nabla \phi(\vec{x}) = \sum_{j \in N} m_j \frac{\phi_j}{\rho_j} \nabla W(\vec{x}_j - \vec{x}, h)$$

Laplacian:

$$\nabla^2 \phi(\vec{x}) = \sum_{j \in N} m_j \frac{\phi_j}{\rho_j} \nabla^2 W(\vec{x}_j - \vec{x}, h)$$

> **Key Insight**
>
> Notice that differential operators (gradient, Laplacian) are applied only to the kernel function, not the physical quantities! This is the genius of SPH—we precompute kernel derivatives analytically.

Different kernel functions are used for different quantities (density, pressure, viscosity) for numerical stability.

Density Calculation

$$\rho(\vec{x}) = \sum_{j \in N} m_j W_{\text{poly6}}(\vec{x}_j - \vec{x}, h)$$

Uses the **Poly6 kernel** for smooth density estimation.

Viscosity Calculation

$$f_i^{\text{visc}} = \mu \sum_{j \in N} m_j \frac{\vec{u}_j - \vec{u}_i}{\rho_j} \nabla^2 W_{\text{visc}}(\vec{x}_j - \vec{x}, h)$$

Uses the **Viscosity kernel's Laplacian**.

Pressure Calculation

$$f_i^{\text{press}} = -\frac{1}{\rho_i} \sum_{j \in N} m_j \frac{p_j + p_i}{2\rho_j} \nabla W_{\text{spiky}}(\vec{x}_j - \vec{x}, h)$$

Uses the **Spiky kernel's gradient** (better handles close particles).

Particle pressure is computed via the **Tait equation**:

$$p = B \left\{ \left(\frac{\rho}{\rho_0} \right)^{\gamma} - 1 \right\}$$

Why Tait Instead of Poisson?

True incompressibility requires solving a Poisson equation—expensive! The Tait equation approximates this much faster, though with some compressibility. This trade-off between accuracy and speed is why SPH is considered weaker at pressure than grid methods.

SPH Implementation

Sample code is in Assets/**SPHFluid** in the repository. This implementation prioritizes simplicity over optimization or numerical stability.

Parameters

```
NumParticleEnum particleNum = NumParticleEnum.NUM_8K;  // Particle count
float smoothlen = 0.012f;              // Particle radius (influence radius h)
float pressureStiffness = 200.0f;      // Pressure coefficient
float restDensity = 1000.0f;           // Rest density
float particleMass = 0.0002f;          // Particle mass
float viscosity = 0.1f;                // Viscosity coefficient
float maxAllowableTimestep = 0.005f;   // Time step
float wallStiffness = 3000.0f;         // Wall repulsion (penalty method)
int iterations = 4;                    // Iterations per frame
Vector2 gravity = new Vector2(0.0f, -0.5f);  // Gravity
Vector2 range = new Vector2(1, 1);           // Simulation space
```

Kernel Coefficient Pre-calculation

Since kernel coefficients don't change during simulation, compute them once on CPU:

```
densityCoef = particleMass * 4f / (Mathf.PI * Mathf.Pow(smoothlen, 8));
gradPressureCoef = particleMass * -30.0f / (Mathf.PI * Mathf.Pow(smoothlen, 5));
lapViscosityCoef = particleMass * 20f / (3 * Mathf.PI * Mathf.Pow(smoothlen, 5));
```

Density Kernel

```
[numthreads(THREAD_SIZE_X, 1, 1)]
void DensityCS(uint3 DTid : SV_DispatchThreadID) {
    uint P_ID = DTid.x;  // Current particle ID

    float h_sq = _Smoothlen * _Smoothlen;
    float2 P_position = _ParticlesBufferRead[P_ID].position;

    // Neighbor search (O(n^2) - simple but slow)
    float density = 0;
    for (uint N_ID = 0; N_ID < _NumParticles; N_ID++) {
        if (N_ID == P_ID) continue;  // Skip self

        float2 N_position = _ParticlesBufferRead[N_ID].position;
        float2 diff = N_position - P_position;
        float r_sq = dot(diff, diff);

        // Only particles within influence radius
        if (r_sq < h_sq) {
```

```
            density += CalculateDensity(r_sq);
        }
    }

    _ParticlesDensityBufferWrite[P_ID].density = density;
}

inline float CalculateDensity(float r_sq) {
    const float h_sq = _Smoothlen * _Smoothlen;
    // Poly6 kernel (only uses squared distances - no sqrt needed!)
    return _DensityCoef * (h_sq - r_sq) * (h_sq - r_sq) * (h_sq - r_sq);
}
```

Performance Note

The O(n^2) neighbor search is simple but slow. Production implementations use spatial hashing or grid-based neighbor finding to achieve O(n) complexity.

Pressure Kernel

```
[numthreads(THREAD_SIZE_X, 1, 1)]
void PressureCS(uint3 DTid : SV_DispatchThreadID) {
    uint P_ID = DTid.x;

    float P_density = _ParticlesDensityBufferRead[P_ID].density;
    float P_pressure = CalculatePressure(P_density);

    _ParticlesPressureBufferWrite[P_ID].pressure = P_pressure;
}

// Tait equation for pressure
inline float CalculatePressure(float density) {
    return _PressureStiffness * max(pow(density / _RestDensity, 7) - 1, 0);
}
```

Force Kernel (Pressure + Viscosity)

```
[numthreads(THREAD_SIZE_X, 1, 1)]
void ForceCS(uint3 DTid : SV_DispatchThreadID) {
    uint P_ID = DTid.x;

    float2 P_position = _ParticlesBufferRead[P_ID].position;
    float2 P_velocity = _ParticlesBufferRead[P_ID].velocity;
    float  P_density = _ParticlesDensityBufferRead[P_ID].density;
    float  P_pressure = _ParticlesPressureBufferRead[P_ID].pressure;

    const float h_sq = _Smoothlen * _Smoothlen;

    float2 press = float2(0, 0);
    float2 visco = float2(0, 0);

    for (uint N_ID = 0; N_ID < _NumParticles; N_ID++) {
        if (N_ID == P_ID) continue;

        float2 N_position = _ParticlesBufferRead[N_ID].position;
```

```
        float2 diff = N_position - P_position;
        float r_sq = dot(diff, diff);

        if (r_sq < h_sq) {
            float N_density = _ParticlesDensityBufferRead[N_ID].density;
            float N_pressure = _ParticlesPressureBufferRead[N_ID].pressure;
            float2 N_velocity = _ParticlesBufferRead[N_ID].velocity;
            float r = sqrt(r_sq);

            // Pressure force (Spiky kernel gradient)
            press += CalculateGradPressure(...);

            // Viscosity force (Viscosity kernel Laplacian)
            visco += CalculateLapVelocity(...);
        }
    }

    float2 force = press + _Viscosity * visco;
    _ParticlesForceBufferWrite[P_ID].acceleration = force / P_density;
}
```

Integration Kernel (Position Update)

```
[numthreads(THREAD_SIZE_X, 1, 1)]
void IntegrateCS(uint3 DTid : SV_DispatchThreadID) {
    const unsigned int P_ID = DTid.x;

    float2 position = _ParticlesBufferRead[P_ID].position;
    float2 velocity = _ParticlesBufferRead[P_ID].velocity;
    float2 acceleration = _ParticlesForceBufferRead[P_ID].acceleration;

    // Mouse interaction
    if (distance(position, _MousePos.xy) < _MouseRadius && _MouseDown) {
        float2 dir = position - _MousePos.xy;
        float pushBack = _MouseRadius - length(dir);
        acceleration += 100 * pushBack * normalize(dir);
    }

    // Wall boundaries (penalty method)
    float dist = dot(float3(position, 1), float3(1, 0, 0));
    acceleration += min(dist, 0) * -_WallStiffness * float2(1, 0);
    // ... (other walls)

    // Add gravity
    acceleration += _Gravity;

    // Forward Euler integration
    velocity += _TimeStep * acceleration;
    position += _TimeStep * velocity;

    _ParticlesBufferWrite[P_ID].position = position;
    _ParticlesBufferWrite[P_ID].velocity = velocity;
}
```

Penalty Method for Walls

When a particle penetrates a wall boundary, push it back with force proportional to penetration depth. Simple and effective for basic boundary handling.

Main Simulation Loop

```
private void RunFluidSolver() {
    int kernelID = -1;
    int threadGroupsX = numParticles / THREAD_SIZE_X;

    // Step 1: Density
    kernelID = fluidCS.FindKernel("DensityCS");
    fluidCS.SetBuffer(kernelID, "_ParticlesBufferRead", ...);
    fluidCS.SetBuffer(kernelID, "_ParticlesDensityBufferWrite", ...);
    fluidCS.Dispatch(kernelID, threadGroupsX, 1, 1);

    // Step 2: Pressure (from density)
    kernelID = fluidCS.FindKernel("PressureCS");
    // ...

    // Step 3: Forces (pressure + viscosity)
    kernelID = fluidCS.FindKernel("ForceCS");
    // ...

    // Step 4: Integrate (update positions)
    kernelID = fluidCS.FindKernel("IntegrateCS");
    // ...

    // Swap read/write buffers
    SwapComputeBuffer(ref particlesBufferRead, ref particlesBufferWrite);
}
```

Sub-stepping for Stability

Smaller timesteps = more accurate simulation. But $\Delta t = 1/60$ (for 60 FPS) is too large—particles explode!

Solution: Run multiple iterations per frame with smaller timesteps:

```
// Run multiple iterations with smaller timestep for stability
for (int i = 0; i < iterations; i++) {
    RunFluidSolver();
}
```

With `iterations = 4` and 60 FPS, effective timestep is $\Delta t = 1/(60 \times 4) = 1/240$.

Double Buffering

Since particles interact with each other, we can't let data change mid-calculation. Use two buffers and swap each frame:

```
void SwapComputeBuffer(ref ComputeBuffer ping, ref ComputeBuffer pong) {
    ComputeBuffer temp = ping;
    ping = pong;
    pong = temp;
}
```

Particle Rendering

Particles are rendered as point sprites (billboards) using geometry shaders:

```
void DrawParticle() {
    RenderParticleMat.SetPass(0);
    RenderParticleMat.SetColor("_WaterColor", WaterColor);
    RenderParticleMat.SetBuffer("_ParticlesBuffer", solver.ParticlesBufferRead);
    Graphics.DrawProceduralNow(MeshTopology.Points, solver.NumParticles);
}
```

The vertex shader reads particle positions from the buffer using `SV_VertexID`, and the geometry shader expands each point into a camera-facing quad (billboard).

Results

The simulation produces interactive 2D fluid that responds to mouse input.

Video: https://youtu.be/KJVu26zeK2w

Summary

This chapter demonstrated SPH fluid simulation. SPH treats fluid as a generic particle system, making it intuitive and parallelizable.

Beyond SPH, many other fluid simulation methods exist. We hope this chapter sparks interest in exploring other physics simulations and expanding your creative toolkit.

Key Takeaways

What You Should Remember

1. **Lagrangian = moving observation points** □ Particles naturally handle advection
2. **Material derivative** $\frac{D}{Dt}$ □ Combines local and convective change
3. **SPH kernel functions** □ Weight contributions by distance, apply derivatives to kernels only
4. **Different kernels for different quantities** □ Poly6 (density), Spiky (pressure), Viscosity (viscosity)
5. **Tait equation** □ Fast pressure approximation (trades accuracy for speed)
6. **Sub-stepping** □ Multiple small timesteps per frame for stability
7. **Double buffering** □ Prevents read/write conflicts in parallel computation

---

SPH vs Grid Methods Comparison

Aspect	Grid (Stable Fluids)	Particle (SPH)
Perspective	Eulerian (fixed points)	Lagrangian (moving points)
Advection	Explicit calculation needed	Automatic (particles move)
Pressure	Accurate (Poisson solver)	Approximate (Tait equation)
Neighbor finding	Trivial (grid structure)	Requires spatial data structure
Free surfaces	Difficult	Natural
Splashing	Difficult	Natural
Memory	Fixed (grid resolution)	Scales with particle count

References

- Desbrun and Cani, “Smoothed Particles: A new paradigm for animating highly deformable bodies,” 1996
- “Fluid Simulation for Computer Graphics” - Robert Bridson

Next chapter: Geometry Shaders for Grass Rendering!

Chapter 6: Growing Grass with Geometry Shaders

Original author: @aoyama Translation and annotations by Claude

---

Introduction

This chapter explains the **Geometry Shader**, one stage of the rendering pipeline, with a focus on creating a dynamic grass generation shader (commonly called a “Grass Shader”).

Some technical terminology is used in the Geometry Shader explanation, but if you just want to try it out, looking at the sample code is the quickest approach.

The Unity project for this chapter is available at: https://github.com/IndieVisualLab/UnityGraphicsProgramming/

What is a Geometry Shader?

A Geometry Shader is a **programmable shader** that can dynamically transform, generate, and delete primitives (the basic shapes that make up meshes) on the GPU.

Previously, to dynamically change mesh shapes, you had to either: - Process on the CPU (slow!) - Pre-encode metadata in vertices and transform in the Vertex Shader (limited)

The Vertex Shader has strong constraints—it cannot access neighboring vertex information, cannot generate new vertices, and cannot delete vertices.

Geometry Shaders solve these problems. Introduced in DirectX 10 and OpenGL 3.2, they allow flexible transformation with fewer constraints.

> **Alternative Names**
>
> In OpenGL, Geometry Shaders are sometimes called “Primitive Shaders.”

Geometry Shader Characteristics

Position in the Rendering Pipeline

`Vertex Shader → Geometry Shader → Rasterization → Fragment Shader`

The Geometry Shader sits between Vertex Shader and Fragment Shader. The Fragment Shader processes vertices generated by the Geometry Shader exactly the same as original vertices—it can't tell the difference.

Input to Geometry Shaders

While Vertex Shaders receive input per-vertex, Geometry Shaders receive input per-**primitive** (user-defined).

The Vertex Shader processes vertices first, then groups them according to the input primitive type: - `triangle`: 3 vertices per invocation - `line`: 2 vertices per invocation - `point`: 1 vertex per invocation

This allows the Geometry Shader to access multiple vertices simultaneously—something impossible in Vertex Shaders!

> **Important Distinction**
>
> The Geometry Shader executes once per **primitive** (determined by mesh topology), not per input type. For a Quad mesh (2 triangles), the Geometry Shader runs twice—once for each triangle—regardless of input primitive type setting.

Output from Geometry Shaders

Output is a stream of vertices that form user-defined output primitives. Unlike Vertex Shaders (1 input 1 output), Geometry Shaders can output **multiple** primitives.

For example, with `triangle` output type and 9 output vertices, you generate 3 triangles. Since this runs per input primitive, one input triangle can become three output triangles.

MaxVertexCount: You must declare the maximum vertices you'll output per invocation. For example, `[maxvertexcount(9)]` means 0-9 vertices can be output.

> **Critical Understanding**
>
> The Geometry Shader **replaces** Vertex Shader output, not appends to it. To keep original geometry, you must explicitly output those vertices too. Conversely, outputting nothing effectively deletes primitives.

Geometry Shader Limitations

Two limits constrain Geometry Shader output:

1. **Max Output Vertex Count**: GPU-dependent, typically 1024
2. **Max Output Element Count**: GPU-dependent, typically 1024

"Elements" means vertex attributes (position xyzw + color rgba = 8 elements). With 8 elements per vertex and 1024 max elements, practical limit is **128 vertices** (1024/8).

Both limits must be satisfied, so the effective maximum is usually 128 vertices per invocation.

Simple Geometry Shader Example

Here's a basic Geometry Shader that transforms flat triangles into pyramids:

```
Shader "Custom/SimpleGeometryShader"
{
    Properties
    {
        _Height("Height", float) = 5.0
        _TopColor("Top Color", Color) = (0.0, 0.0, 1.0, 1.0)
        _BottomColor("Bottom Color", Color) = (1.0, 0.0, 0.0, 1.0)
    }
    SubShader
    {
        Tags { "RenderType" = "Opaque"}
        LOD 100
        Cull Off
        Lighting Off

        Pass
        {
            CGPROGRAM
            #pragma target 5.0
            #pragma vertex vert
            #pragma geometry geom
            #pragma fragment frag
            #include "UnityCG.cginc"

            uniform float _Height;
            uniform float4 _TopColor, _BottomColor;

            struct v2g
            {
                float4 pos : SV_POSITION;
            };

            struct g2f
            {
                float4 pos : SV_POSITION;
                float4 col : COLOR;
            };

            v2g vert(appdata_full v)
            {
                v2g o;
                o.pos = v.vertex;  // Pass through in object space
                return o;
            }

            [maxvertexcount(12)]
            void geom(triangle v2g input[3],
                      inout TriangleStream<g2f> outStream)
            {
                float4 p0 = input[0].pos;
                float4 p1 = input[1].pos;
                float4 p2 = input[2].pos;

                // Calculate pyramid apex above triangle center
```

```
                float4 c = float4(0.0f, 0.0f, -_Height, 1.0f)
                            + (p0 + p1 + p2) * 0.33333f;

                // Create output vertices
                g2f out0, out1, out2, o;
                out0.pos = UnityObjectToClipPos(p0);
                out0.col = _BottomColor;
                out1.pos = UnityObjectToClipPos(p1);
                out1.col = _BottomColor;
                out2.pos = UnityObjectToClipPos(p2);
                out2.col = _BottomColor;
                o.pos = UnityObjectToClipPos(c);
                o.col = _TopColor;

                // Bottom triangle
                outStream.Append(out0);
                outStream.Append(out1);
                outStream.Append(out2);
                outStream.RestartStrip();

                // Side triangles
                outStream.Append(out0);
                outStream.Append(out1);
                outStream.Append(o);
                outStream.RestartStrip();

                outStream.Append(out1);
                outStream.Append(out2);
                outStream.Append(o);
                outStream.RestartStrip();

                outStream.Append(out2);
                outStream.Append(out0);
                outStream.Append(o);
                outStream.RestartStrip();
            }

            float4 frag(g2f i) : COLOR
            {
                return i.col;
            }
            ENDCG
        }
    }
}
```

This transforms each input triangle into a 4-sided pyramid (1 bottom + 3 sides = 4 triangles = 12 vertices).

Key Parts Explained

Declaration:

`#pragma geometry geom`

Declares `geom` as the Geometry Shader function.

Function signature:

```
[maxvertexcount(12)]
void geom(triangle v2g input[3], inout TriangleStream<g2f> outStream)
```

Input: `triangle v2g input[3]` - `triangle` = input primitive type (3 vertices) - `v2g` = vertex-to-geometry struct - `input[3]` = array of 3 vertices

Output: `inout TriangleStream<g2f> outStream` - `TriangleStream` = output as triangle strips - Other options: `PointStream`, `LineStream`

Output methods: - `Append()` = add vertex to current strip - `RestartStrip()` = end current strip, start new one

> **Triangle Strips vs Separate Triangles**
>
> `TriangleStream` creates triangle strips—each new vertex after the first 3 forms a new triangle with the previous 2. Use `RestartStrip()` when triangles shouldn't connect.

Grass Shader

Now let's build on these concepts to create dynamic grass:

```
Shader "Custom/Grass" {
    Properties
    {
        _Height("Height", float) = 80
        _Width("Width", float) = 2.5

        _BottomHeight("Bottom Height", float) = 0.3
        _MiddleHeight("Middle Height", float) = 0.4
        _TopHeight("Top Height", float) = 0.5

        _BottomWidth("Bottom Width", float) = 0.5
        _MiddleWidth("Middle Width", float) = 0.4
        _TopWidth("Top Width", float) = 0.2

        _BottomBend("Bottom Bend", float) = 1.0
        _MiddleBend("Middle Bend", float) = 1.0
        _TopBend("Top Bend", float) = 2.0

        _WindPower("Wind Power", float) = 1.0

        _TopColor("Top Color", Color) = (1.0, 1.0, 1.0, 1.0)
        _BottomColor("Bottom Color", Color) = (0.0, 0.0, 0.0, 1.0)

        _HeightMap("Height Map", 2D) = "white"
        _RotationMap("Rotation Map", 2D) = "black"
        _WindMap("Wind Map", 2D) = "black"
    }
    SubShader
    {
        Tags{ "RenderType" = "Opaque" }
        LOD 100
        Cull Off
```

```
Pass
{
    CGPROGRAM
    #pragma target 5.0
    #include "UnityCG.cginc"

    #pragma vertex vert
    #pragma geometry geom
    #pragma fragment frag

    // ... (property declarations)

    struct v2g
    {
        float4 pos : SV_POSITION;
        float3 nor : NORMAL;
        float4 hei : TEXCOORD0;   // Height from noise texture
        float4 rot : TEXCOORD1;   // Rotation from noise texture
        float4 wind : TEXCOORD2;  // Wind strength from noise texture
    };

    struct g2f
    {
        float4 pos : SV_POSITION;
        float4 color : COLOR;
    };

    v2g vert(appdata_full v)
    {
        v2g o;
        float4 uv = float4(v.texcoord.xy, 0.0f, 0.0f);

        o.pos = v.vertex;
        o.nor = v.normal;
        o.hei = tex2Dlod(_HeightMap, uv);
        o.rot = tex2Dlod(_RotationMap, uv);
        o.wind = tex2Dlod(_WindMap, uv);

        return o;
    }

    [maxvertexcount(7)]
    void geom(triangle v2g i[3], inout TriangleStream<g2f> stream)
    {
        // Get triangle vertices and normals
        float4 p0 = i[0].pos, p1 = i[1].pos, p2 = i[2].pos;
        float3 n0 = i[0].nor, n1 = i[1].nor, n2 = i[2].nor;

        // Average noise values across triangle
        float height = (i[0].hei.r + i[1].hei.r + i[2].hei.r) / 3.0f;
        float rot = (i[0].rot.r + i[1].rot.r + i[2].rot.r) / 3.0f;
        float wind = (i[0].wind.r + i[1].wind.r + i[2].wind.r) / 3.0f;

        // Triangle center and average normal
```

```
float4 center = (p0 + p1 + p2) / 3.0f;
float4 normal = float4(((n0 + n1 + n2) / 3.0f).xyz, 1.0f);

// Calculate segment heights and widths
float bottomHeight = height * _Height * _BottomHeight;
float middleHeight = height * _Height * _MiddleHeight;
float topHeight = height * _Height * _TopHeight;

float bottomWidth = _Width * _BottomWidth;
float middleWidth = _Width * _MiddleWidth;
float topWidth = _Width * _TopWidth;

// Direction for grass blade orientation
rot = rot - 0.5f;
float4 dir = float4(normalize((p2 - p0) * rot).xyz, 1.0f);

g2f o[7];

// Bottom vertices (ground level)
o[0].pos = center - dir * bottomWidth;
o[0].color = _BottomColor;
o[1].pos = center + dir * bottomWidth;
o[1].color = _BottomColor;

// Lower-middle vertices
o[2].pos = center - dir * middleWidth + normal * bottomHeight;
o[2].color = lerp(_BottomColor, _TopColor, 0.33333f);
o[3].pos = center + dir * middleWidth + normal * bottomHeight;
o[3].color = lerp(_BottomColor, _TopColor, 0.33333f);

// Upper-middle vertices
o[4].pos = o[3].pos - dir * topWidth + normal * middleHeight;
o[4].color = lerp(_BottomColor, _TopColor, 0.66666f);
o[5].pos = o[3].pos + dir * topWidth + normal * middleHeight;
o[5].color = lerp(_BottomColor, _TopColor, 0.66666f);

// Top vertex
o[6].pos = o[5].pos + dir * topWidth + normal * topHeight;
o[6].color = _TopColor;

// Apply wind bending (more at top, less at bottom)
dir = float4(1.0f, 0.0f, 0.0f, 1.0f);
o[2].pos += dir * (_WindPower * wind * _BottomBend) * sin(_Time);
o[3].pos += dir * (_WindPower * wind * _BottomBend) * sin(_Time);
o[4].pos += dir * (_WindPower * wind * _MiddleBend) * sin(_Time);
o[5].pos += dir * (_WindPower * wind * _MiddleBend) * sin(_Time);
o[6].pos += dir * (_WindPower * wind * _TopBend) * sin(_Time);

// Output all vertices (as connected triangle strip)
[unroll]
for (int j = 0; j < 7; j++) {
    o[j].pos = UnityObjectToClipPos(o[j].pos);
    stream.Append(o[j]);
}
```

```
            }

            float4 frag(g2f i) : COLOR
            {
                return i.color;
            }
            ENDCG
        }
    }
}
```

Grass Blade Structure

Each grass blade is built from 7 vertices in 3 sections:

```
        6 (top)
       /|\
      5 | 4  (upper-middle)
       \|/
      3 | 2  (lower-middle)
       \|/
      1   0  (bottom/ground)
```

The blade tapers toward the top and bends more with wind at higher positions.

Key Implementation Details

Noise textures for variation: - `_HeightMap`: Varies grass height - `_RotationMap`: Varies grass orientation - `_WindMap`: Varies wind response

Sampling these in the Vertex Shader (via `tex2Dlod`) and passing to Geometry Shader gives each blade unique characteristics.

Wind animation:

```
o[6].pos += dir * (_WindPower * wind * _TopBend) * sin(_Time);
```

Higher vertices bend more, creating natural swaying motion.

Unrolling the loop:

```
[unroll]
for (int j = 0; j < 7; j++) { ... }
```

The `[unroll]` attribute tells the compiler to expand the loop at compile time. Uses more memory but runs faster—worthwhile for small, fixed-iteration loops.

Summary

This chapter covered Geometry Shaders from basics to practical grass generation. While the programming style differs somewhat from CPU code, mastering the fundamentals opens many possibilities.

Performance Note

Geometry Shaders have a reputation for being slow. While the author hasn't experienced significant issues, large-scale use may require benchmarking.

The Bigger Picture

Despite potential performance concerns, the ability to dynamically create, transform, and delete geometry on the GPU dramatically expands creative possibilities. What matters most isn't which technology you use, but what you create and express with it.

Key Takeaways

What You Should Remember

1. **Geometry Shader position**: Between Vertex and Fragment shaders
2. **Input per primitive**: Triangle (3 vertices), Line (2), Point (1)
3. **Output limits**: ~128 vertices per invocation (due to element count limits)
4. **Append & RestartStrip**: Build output geometry vertex by vertex
5. **Triangle Strips**: Efficient for connected triangles; use RestartStrip for disconnected
6. **[unroll]**: Compiler hint for faster small loops
7. **Noise textures**: Add variation for natural-looking results

---

Geometry Shader Use Cases

Use Case	Description
Grass/Foliage	Generate blades from terrain triangles
Particle Expansion	Expand points to billboards
Wireframe Rendering	Generate lines from triangles
Normal Visualization	Draw debug normals
Tessellation	Subdivide geometry
Shadow Volumes	Extrude silhouettes
Fur/Hair	Generate strands from surface

References

- Geometry Shader Tutorial - MSDN
- Geometry Shader Object - MSDN
- Rendering Techniques for Transparent Geometry using Geometry Shader Geometry Cutting

---

Next chapter: Marching Cubes for Isosurface Extraction!

Chapter 7: Introduction to Marching Cubes

Original author: @kaiware007 Translation and annotations by Claude

---

What is Marching Cubes?

History and Overview

Marching Cubes is a volume rendering algorithm that converts 3D voxel data (filled with scalar values) into polygon mesh data. It was first published by William E. Lorensen and Harvey E. Cline in 1987.

The algorithm was patented, but the patent expired in 2005, so it's now freely usable.

> **Where You've Seen It**
>
> Marching Cubes is used in: - Medical imaging (CT/MRI visualization) - Terrain generation with destructible voxels (games like Astroneer) - Fluid surface extraction - 3D scanning/reconstruction - Metaball rendering

How It Works (Simplified)

Step 1: Divide the volume data space into a 3D grid.

Step 2: For each grid cell (cube), examine the 8 corner values. If a corner's value is above the threshold, mark it as 1; otherwise 0.

Step 3: The 8 corners give 2^8 = 256 possible combinations. Through rotation and reflection symmetry, these reduce to just **15 unique patterns**.

Step 4: Look up the triangle pattern for this combination and generate the corresponding polygons.

> **Why "Marching"?**
>
> The algorithm "marches" through the volume, processing one cube at a time, hence the name. Each cube is processed independently, making it highly parallelizable on GPUs.

Sample Repository

The sample project is in **Assets/GPUMarchingCubes** at: https://github.com/IndieVisualLab/UnityGraphicsProgramming

Implementation is based on Paul Bourke's "Polygonising a scalar field" article.

The implementation has three main parts: 1. Mesh initialization and per-frame rendering (C# scripts) 2. ComputeBuffer initialization 3. Actual rendering (shaders)

Creating Geometry Shader-Compatible Meshes

As explained, Marching Cubes generates polygons based on the 8 corners of each grid cell. For real-time rendering, we need dynamic polygon generation.

Creating mesh vertex arrays on the CPU every frame would be inefficient. Instead, we use **Geometry Shaders**—they sit between Vertex and Fragment shaders and can dynamically create/modify vertices on the GPU, making them very fast.

Variable Definitions

```
public class GPUMarchingCubesDrawMesh : MonoBehaviour {

    #region public
    public int segmentNum = 32;              // Grid divisions per axis
    [Range(0,1)]
```

```
    public float threashold = 0.5f;          // Threshold for mesh generation
    public Material mat;                     // Rendering material

    public Color DiffuseColor = Color.green;
    public Color EmissionColor = Color.black;
    public float EmissionIntensity = 0;

    [Range(0,1)]
    public float metallic = 0;
    [Range(0, 1)]
    public float glossiness = 0.5f;
    #endregion

    #region private
    int vertexMax = 0;                       // Total vertex count
    Mesh[] meshs = null;                     // Mesh array
    Material[] materials = null;             // Per-mesh materials
    float renderScale = 1f / 32f;            // Display scale
    MarchingCubesDefines mcDefines = null;   // MC lookup tables
    #endregion
}
```

Mesh Creation

Vertices are placed one per grid cell. For 64 divisions per axis: 64×64×64 = 262,144 vertices needed.

However, Unity (2017.1.1f1) limits meshes to 65,535 vertices. So we split into multiple meshes:

```
void CreateMesh()
{
    // Split meshes to stay under 65535 vertex limit
    int vertNum = 65535;
    int meshNum = Mathf.CeilToInt((float)vertexMax / vertNum);

    meshs = new Mesh[meshNum];
    materials = new Material[meshNum];

    // Calculate bounds
    Bounds bounds = new Bounds(
        transform.position,
        new Vector3(segmentNum, segmentNum, segmentNum) * renderScale
    );

    int id = 0;
    for (int i = 0; i < meshNum; i++)
    {
        Vector3[] vertices = new Vector3[vertNum];
        int[] indices = new int[vertNum];

        for(int j = 0; j < vertNum; j++)
        {
            // Encode 3D grid position in vertex coordinates
            vertices[j].x = id % segmentNum;
            vertices[j].y = (id / segmentNum) % segmentNum;
            vertices[j].z = (id / (segmentNum * segmentNum)) % segmentNum;
```

```
            indices[j] = j;
            id++;
        }

        meshs[i] = new Mesh();
        meshs[i].vertices = vertices;
        // Use Points topology - Geometry Shader will create actual polygons
        meshs[i].SetIndices(indices, MeshTopology.Points, 0);
        meshs[i].bounds = bounds;

        materials[i] = new Material(mat);
    }
}
```

Key Insight

Each vertex represents a grid cell position. The Geometry Shader will read the scalar field at each cell's 8 corners and generate triangles accordingly. Using `MeshTopology.Points` means we're just passing positions—the shader does the rest.

ComputeBuffer Initialization

The `MarchingCubesDefines.cs` file contains lookup tables and ComputeBuffers for the Marching Cubes algorithm.

Why ComputeBuffers instead of shader constants?

Shaders have a limit of ~4096 literal values. The Marching Cubes lookup tables are huge and would exceed this limit. By storing them in ComputeBuffers (GPU memory), we avoid this restriction and get fast shader access.

Rendering

```
void RenderMesh()
{
    Vector3 halfSize = new Vector3(segmentNum, segmentNum, segmentNum)
                       * renderScale * 0.5f;
    Matrix4x4 trs = Matrix4x4.TRS(
                       transform.position,
                       transform.rotation,
                       transform.localScale
                    );

    for (int i = 0; i < meshs.Length; i++)
    {
        materials[i].SetPass(0);
        materials[i].SetInt("_SegmentNum", segmentNum);
        materials[i].SetFloat("_Scale", renderScale);
        materials[i].SetFloat("_Threashold", threashold);
        // ... (other parameters)

        Graphics.DrawMesh(meshs[i], Matrix4x4.identity, materials[i], 0);
```

```
    }
}
```

We use `Graphics.DrawMesh()` for Unity lighting integration. This registers the mesh for rendering (doesn't draw immediately), allowing Unity to apply lights and shadows.

DrawMesh vs DrawMeshNow

- `DrawMesh()`: Registers for rendering, Unity handles lighting/shadows, call in Update()
- `DrawMeshNow()`: Immediate draw, no Unity lighting, call in OnRenderObject()/OnPostRender()

Shader Implementation

The shader has two main parts: **object rendering** and **shadow rendering**. Each uses vertex, geometry, and fragment shaders.

Data Structures

```
// Mesh input data
struct appdata
{
    float4 vertex : POSITION;
};

// Vertex → Geometry
struct v2g
{
    float4 pos : SV_POSITION;
};

// Geometry → Fragment (object rendering)
struct g2f_light
{
    float4 pos      : SV_POSITION;  // Clip space position
    float3 normal   : NORMAL;
    float4 worldPos : TEXCOORD0;
    half3 sh        : TEXCOORD3;    // Spherical harmonics
};

// Geometry → Fragment (shadow rendering)
struct g2f_shadow
{
    float4 pos  : SV_POSITION;
    float4 hpos : TEXCOORD1;
};
```

Shader Variables

```
int _SegmentNum;
float _Scale;
float _Threashold;
float4 _DiffuseColor;
// ... (other parameters)
```

```
// Lookup tables from ComputeBuffers
StructuredBuffer<float3> vertexOffset;
StructuredBuffer<int> cubeEdgeFlags;
StructuredBuffer<int2> edgeConnection;
StructuredBuffer<float3> edgeDirection;
StructuredBuffer<int> triangleConnectionTable;
```

Vertex Shader

Very simple—just passes vertex data to geometry shader:

```
v2g vert(appdata v)
{
    v2g o = (v2g)0;
    o.pos = v.vertex;  // Grid position encoded in vertex
    return o;
}
```

Geometry Shader (Object)

```
[maxvertexcount(15)]  // Max 5 triangles × 3 vertices = 15
void geom_light(point v2g input[1],
                inout TriangleStream<g2f_light> outStream)
{
    float3 pos = input[0].pos.xyz;
    float cubeValue[8];

    // Sample scalar field at 8 cube corners
    for (int i = 0; i < 8; i++) {
        cubeValue[i] = Sample(
            pos.x + vertexOffset[i].x,
            pos.y + vertexOffset[i].y,
            pos.z + vertexOffset[i].z
        );
    }

    // Build flag index from corner values vs threshold
    int flagIndex = 0;
    for (i = 0; i < 8; i++) {
        if (cubeValue[i] <= _Threashold) {
            flagIndex |= (1 << i);
        }
    }

    int edgeFlags = cubeEdgeFlags[flagIndex];

    // Empty or full cube - no surface
    if ((edgeFlags == 0) || (edgeFlags == 255)) {
        return;
    }

    // Calculate edge vertices where surface intersects cube edges
    float3 edgeVertices[12];
    float3 edgeNormals[12];
```

```
    for (i = 0; i < 12; i++) {
        if ((edgeFlags & (1 << i)) != 0) {
            // Interpolate along edge based on threshold
            float offset = getOffset(
                cubeValue[edgeConnection[i].x],
                cubeValue[edgeConnection[i].y],
                _Threashold
            );

            float3 vertex = vertexOffset[edgeConnection[i].x]
                          + offset * edgeDirection[i];

            edgeVertices[i] = pos + vertex * _Scale;
            edgeNormals[i] = getNormal(pos + vertex);
        }
    }

    // Generate triangles from lookup table
    for (i = 0; i < 5; i++) {  // Max 5 triangles
        int findex = flagIndex * 16 + 3 * i;
        if (triangleConnectionTable[findex] < 0)
            break;

        for (int j = 0; j < 3; j++) {
            int vindex = triangleConnectionTable[findex + j];

            g2f_light o;
            float4 ppos = mul(_Matrix, float4(edgeVertices[vindex], 1));
            o.pos = UnityObjectToClipPos(ppos);
            o.normal = normalize(mul(_Matrix, float4(edgeNormals[vindex], 0)));
            o.worldPos = ppos;

            outStream.Append(o);
        }
        outStream.RestartStrip();
    }
}
```

Understanding the Algorithm

1. Sample 8 corners 8 scalar values
2. Compare to threshold 8-bit flag (256 possibilities)
3. Look up edge flags which of 12 edges are crossed
4. For each crossed edge, interpolate vertex position
5. Look up triangle table which edge vertices connect
6. Output triangles

Distance Functions

Instead of sampling from actual volume data, this example uses **distance functions** to define shapes procedurally.

```
// Sphere distance function
inline float sphere(float3 pos, float radius)
{
    return length(pos) - radius;
}
```

If `sphere()` returns negative, the point is inside the sphere. This simple formula defines complex 3D shapes with minimal code.

```
// Smooth blending of distance functions (metaball effect)
float smoothMax(float d1, float d2, float k)
{
    float h = exp(k * d1) + exp(k * d2);
    return log(h) / k;
}
```

The `smoothMax` function blends multiple shapes smoothly, creating metaball-like organic forms.

> **Distance Function Resources**
>
> Inigo Quilez's website has an excellent collection: http://iquilezles.org/www/articles/distfunctions/distfunctions.htm

---

Fragment Shader (Object)

Uses Unity's Standard lighting with deferred rendering:

```
void frag_light(g2f_light IN,
    out half4 outDiffuse        : SV_Target0,
    out half4 outSpecSmoothness : SV_Target1,
    out half4 outNormal         : SV_Target2,
    out half4 outEmission       : SV_Target3)
{
    // Initialize surface output
    SurfaceOutputStandard o;
    o.Albedo = _DiffuseColor.rgb;
    o.Emission = _EmissionColor * _EmissionIntensity;
    o.Metallic = _Metallic;
    o.Smoothness = _Glossiness;
    o.Normal = IN.normal;
    // ...

    // Setup GI
    UnityGI gi;
    UnityGIInput giInput;
    // ... (GI setup code)

    LightingStandard_GI(o, giInput, gi);

    // Output to G-buffer
    outEmission = LightingStandard_Deferred(o, worldViewDir, gi,
                                            outDiffuse,
                                            outSpecSmoothness,
                                            outNormal);
}
```

This integrates with Unity's deferred rendering for proper lighting, reflections, and GI.

Shadow Shaders

The shadow geometry shader is nearly identical to the object version, but outputs to shadow maps:

```
// Shadow vertex transformation
o.pos = UnityClipSpaceShadowCasterPos(lpos, normal);
o.pos = UnityApplyLinearShadowBias(o.pos);
```

The shadow fragment shader is trivial—Unity handles the heavy lifting:

```
fixed4 frag_shadow(g2f_shadow i) : SV_Target
{
    return i.hpos.z / i.hpos.w;
}
```

Results

Running the sample produces animated metaball-like shapes:

The distance functions can be combined to create various forms—the sample shows animated spheres that merge organically.

Summary

This chapter used distance functions for simplicity, but Marching Cubes works with any 3D scalar data: - 3D textures with volume data - Medical imaging data (CT/MRI) - Procedural terrain

For games, you could create destructible/constructible terrain like Astroneer by modifying the underlying volume data.

Key Takeaways

What You Should Remember

1. **Marching Cubes**: Converts volume data to polygons by examining cube corners
2. **256 → 15 patterns**: Symmetry reduces lookup table complexity
3. **Geometry Shader**: Creates triangles dynamically on GPU
4. **ComputeBuffers**: Store large lookup tables (bypass shader literal limits)
5. **Distance functions**: Define shapes procedurally with simple math
6. **Edge interpolation**: Place vertices where surface crosses cube edges
7. **MeshTopology.Points**: Pass grid positions, geometry shader generates actual geometry

---

The 15 Marching Cubes Cases

Case	Description
0	All corners outside (empty)
1	One corner inside (single triangle)
2	Two adjacent corners (quad)
3	Two diagonal corners
4	Three corners (various configs)
…	…
14	Seven corners inside
15	All corners inside (full, no surface)

Each case has a predefined triangle configuration stored in the lookup table.

References

- Polygonising a scalar field - http://paulbourke.net/geometry/polygonise/
- Distance functions - http://iquilezles.org/www/articles/distfunctions/distfunctions.htm

---

Next chapter: MCMC 3D Sampling!

Chapter 8: MCMC 3D Space Sampling

Original author: @komietty Translation and annotations by Claude

---

Introduction

This chapter explains **MCMC (Markov Chain Monte Carlo)**, a sampling method that draws multiple values from a given probability distribution.

The simplest method for sampling from a probability distribution is **rejection sampling**, but in 3D space, the rejection region becomes very large, making it impractical. MCMC enables efficient sampling even in high-dimensional spaces.

> **The Information Gap**
>
> MCMC information tends to fall into two camps: academic books with rigorous theory but no implementation guidance, or online code snippets with no theoretical background. This chapter aims to bridge that gap—providing enough theory to understand what you're implementing, without getting lost in mathematical formalism.

The probability concepts needed here are kept minimal. We'll aim for intuitive understanding rather than mathematical rigor.

Sample Repository

Sample code is in **Assets/MCMC3D** at: https://github.com/IndieVisualLab/UnityGraphicsProgramming

---

Probability Fundamentals

To understand MCMC, you only need four concepts:

1. Random Variable
2. Probability Distribution
3. Stochastic Process
4. Stationary Distribution

Random Variable

When an event occurs with probability P(X), the value X is called a **random variable**. For example: "The probability of rolling a 5 on a die is 1/6"—here "5" is the random variable and "1/6" is the probability.

More formally: a random variable X is a mapping $X = X(\omega)$ from the sample space Ω (all possible outcomes) to real numbers.

Stochastic Process

A **stochastic process** adds a time dimension to random variables: $X = X(\omega, t)$. It's essentially a random variable that evolves over time.

Probability Distribution

A **probability distribution** shows the relationship between random variable X and probability P(X). Often visualized as a graph with X on the horizontal axis and P(X) on the vertical axis.

Stationary Distribution

A **stationary distribution** is one where individual points may transition, but the overall distribution shape remains unchanged. For distribution P and transition matrix π, if $\pi P = P$, then P is a stationary distribution.

> **Visual Intuition**
>
> Imagine particles bouncing around according to some rules. Even though each particle moves, if the overall "cloud" of particles maintains the same shape, that shape is the stationary distribution.

MCMC Concepts

MCMC samples from a given distribution that is assumed to be stationary, using **Monte Carlo** methods combined with **Markov chains**.

Monte Carlo Methods

Monte Carlo methods use pseudo-random numbers for numerical computation or simulation.

A classic example—estimating π:

```
float pi;
float trial = 10000;
float count = 0;

for(int i = 0; i < trial; i++){
    float x = Random.value;
```

```
    float y = Random.value;
    if(x*x + y*y <= 1) count++;
}

pi = 4 * count / trial;
```

This randomly samples points in a unit square and counts how many fall inside a quarter circle. The ratio approximates π/4.

Markov Chains

A **Markov chain** is a stochastic process where **future states depend only on the current state, not on past history**. This is called the **Markov property**.

> **"Memoryless"**
>
> The Markov property means the system has no memory. Where you came from doesn't matter—only where you are now determines where you can go next.

Convergence to Stationary Distribution

For MCMC to work, the sampling process must converge to the target distribution. This requires two conditions:

1. Irreducibility: The distribution cannot be split into disconnected parts. From any point, you must be able to reach any other point through transitions.

2. Aperiodicity: You can return to any state in any number of steps. No periodic patterns like "can only return in even numbers of steps."

When both conditions are met, any initial distribution converges to the target stationary distribution. This is called **ergodicity**.

The Metropolis Algorithm

Checking ergodicity directly is tedious, so we use a stronger condition called **detailed balance**. The Metropolis algorithm is one method that satisfies detailed balance.

Metropolis algorithm steps:

1. **Propose**: Generate a candidate next state x' using a proposal distribution Q where Q(x|x') = Q(x'|x) (symmetric). Gaussian distributions are commonly used.
2. **Accept/Reject**: Generate a uniform random number r [0,1). If P(x')/P(x) > r, accept the transition to x'. Otherwise, stay at x.

   > **Why This Works**
   >
   > The acceptance criterion means: - Higher probability regions are always accepted - Lower probability regions are accepted with probability proportional to the ratio
   >
   > This lets the sampler explore the distribution while spending more time in high-probability regions—exactly what we want for sampling.

The Metropolis algorithm is a special case of **Metropolis-Hastings** that uses symmetric proposal distributions.

3D Sampling Implementation

Let's implement MCMC for 3D space sampling.

Creating the Target Distribution

First, create a 3D probability distribution to sample from:

```
void Prepare()
{
    var sn = new SimplexNoiseGenerator();
    for (int x = 0; x < lEdge; x++)
        for (int y = 0; y < lEdge; y++)
            for (int z = 0; z < lEdge; z++)
            {
                var i = x + lEdge * y + lEdge * lEdge * z;
                var val = sn.noise(x, y, z);
                data[i] = new Vector4(x, y, z, val);
            }
}
```

This example uses **Simplex noise** as the target distribution—a continuous 3D scalar field with varying density.

Running MCMC

```
public IEnumerable<Vector3> Sequence(int nInit, int limit, float th)
{
    Reset();

    // Burn-in: discard initial samples (not yet converged)
    for (var i = 0; i < nInit; i++)
        Next(th);

    // Main sampling loop
    for (var i = 0; i < limit; i++)
    {
        yield return _curr;
        Next(th);
    }
}

public void Reset()
{
    // Find a valid starting point
    for (var i = 0; _currDensity <= 0f && i < limitResetLoopCount; i++)
    {
        _curr = new Vector3(
            Scale.x * Random.value,
            Scale.y * Random.value,
            Scale.z * Random.value
        );
        _currDensity = Density(_curr);
    }
}
```

Using coroutines allows the process to run across frames. MCMC can be thought of as conceptually parallel—each chain explores independently.

> **Burn-in Period**
>
> Early samples may be far from the target distribution. We discard the first `nInit` samples (burn-in) to let the chain converge before collecting actual samples.

Proposal Distribution

For 3D sampling, use a trivariate standard normal distribution:

```
public static Vector3 GenerateRandomPointStandard()
{
    var x = RandomGenerator.rand_gaussian(0f, 1f);
    var y = RandomGenerator.rand_gaussian(0f, 1f);
    var z = RandomGenerator.rand_gaussian(0f, 1f);
    return new Vector3(x, y, z);
}

public static float rand_gaussian(float mu, float sigma)
{
    // Box–Muller transform for Gaussian random numbers
    float z = Mathf.Sqrt(-2.0f * Mathf.Log(Random.value))
              * Mathf.Sin(2.0f * Mathf.PI * Random.value);
    return mu + sigma * z;
}
```

For non-standard covariance, use Cholesky decomposition:

```
public static Vector3 GenerateRandomPoint(Matrix4x4 sigma)
{
    // Cholesky decomposition for correlated Gaussian
    var c00 = sigma.m00 / Mathf.Sqrt(sigma.m00);
    var c10 = sigma.m10 / Mathf.Sqrt(sigma.m00);
    // ... (decomposition continues)

    var r1 = RandomGenerator.rand_gaussian(0f, 1f);
    var r2 = RandomGenerator.rand_gaussian(0f, 1f);
    var r3 = RandomGenerator.rand_gaussian(0f, 1f);

    var x = c00 * r1;
    var y = c10 * r1 + c11 * r2;
    var z = c20 * r1 + c21 * r2 + c22 * r3;
    return new Vector3(x, y, z);
}
```

Transition Decision

The core Metropolis step:

```
void Next(float threshold)
{
    // Propose: current position + Gaussian offset
    Vector3 next = GaussianDistributionCubic.GenerateRandomPointStandard()
                   + _curr;
```

```
    var densityNext = Density(next);

    // Accept if: (1) current density is zero, OR
    //            (2) ratio exceeds random threshold
    bool flag1 = _currDensity <= 0f ||
                 Mathf.Min(1f, densityNext / _currDensity) >= Random.value;

    // Also require minimum density threshold
    bool flag2 = densityNext > threshold;

    if (flag1 && flag2)
    {
        _curr = next;
        _currDensity = densityNext;
    }
}
```

Density Estimation

Looking up exact density at arbitrary coordinates in a discrete grid is expensive ($O(n^3)$). Instead, approximate using weighted average of nearby samples:

```
float Density(Vector3 pos)
{
    float weight = 0f;
    for (int i = 0; i < weightReferenceloopCount; i++)
    {
        // Random sampling from data array
        int id = (int)Mathf.Floor(Random.value * (Data.Length - 1));
        Vector3 posi = Data[id];
        float mag = Vector3.SqrMagnitude(pos - posi);

        // Inverse distance weighting (exponential falloff)
        weight += Mathf.Exp(-mag) * Data[id].w;
    }
    return weight;
}
```

This uses stochastic sampling with exponential distance weighting for fast approximation.

Additional Notes

The repository includes a **rejection sampling** implementation for comparison. With rejection sampling, strong threshold values cause most samples to be rejected. MCMC produces similar results much more smoothly.

Step size control: By reducing the random walk step size, consecutive samples stay close together. This can be used to simulate clustered phenomena like plant or flower colonies.

Summary

MCMC enables efficient sampling from complex probability distributions in high-dimensional spaces. The key insights are:

1. Markov chains explore the space with “no memory”
2. Detailed balance ensures convergence to target distribution
3. Metropolis acceptance criterion balances exploration and exploitation
4. Burn-in discards non-converged initial samples

Key Takeaways

What You Should Remember

1. **MCMC = Monte Carlo + Markov Chain**: Random sampling with memoryless transitions
2. **Markov property**: Future depends only on present, not past
3. **Stationary distribution**: Overall shape stays constant despite individual transitions
4. **Ergodicity**: Irreducibility + Aperiodicity convergence guaranteed
5. **Metropolis algorithm**: Propose Accept/Reject based on probability ratio
6. **Burn-in**: Discard early samples before convergence
7. **Step size**: Smaller steps clustered samples; larger steps more exploration

---

MCMC vs Rejection Sampling

Aspect	Rejection Sampling	MCMC
Efficiency	Low in high dimensions	Scales well
Correlation	Independent samples	Correlated (chain)
Convergence	N/A	Needs burn-in
Implementation	Simple	More complex
Use case	Low dimensions	High dimensions

Applications

MCMC is used in: - Bayesian inference - Physics simulations - Procedural content generation - Machine learning - Financial modeling - Molecular dynamics

---

References

- Kubo Takuya (2012) “Introduction to Statistical Modeling for Data Analysis”
- Olle Häggström (2017) “Gentle Introduction to MCMC”

Next chapter: Procedural Modeling!

Chapter 9: Introduction to Procedural Modeling in Unity

Original author: @mattatz Translation and annotations by Claude

---

Introduction

Procedural Modeling is a technique for constructing 3D models using rules and algorithms rather than manual manipulation.

Traditional modeling involves using software like Blender or 3ds Max to manually manipulate vertices and edges to achieve a desired shape. In contrast, procedural modeling describes rules that automatically generate shapes through a series of computations.

> **Why Procedural Modeling Matters**
>
> Procedural modeling is used across many domains: - **Games**: Terrain generation, vegetation, city building (enabling stages that change with each playthrough) - **Architecture**: Grasshopper plugin for Rhinoceros CAD enables parametric design - **Film/VFX**: Generating crowds, forests, cities at scale - **3D Printing**: Creating complex structures impossible to model manually

Procedural modeling enables two key capabilities:

1. Parametric Structures

A parametric structure can be transformed by adjusting parameters. For example, a sphere model can have: - **radius**: Controls size - **segments**: Controls smoothness

Once you implement a parametric structure, you can instantly generate variations for any use case.

2. Runtime-Generated Content

Rather than importing pre-built models, procedural techniques can generate models in real-time: - Trees that grow toward the sun at any position - Cities that build up as the user clicks locations - Reduced data size (generate variations instead of storing them)

Master procedural modeling, and you can even build your own modeling tools!

Sample Repository

Sample code is in **Assets/ProceduralModeling** at: https://github.com/IndieVisualLab/UnityGraphicsProgramming

The C# scripts in `Assets/ProceduralModeling/Scripts` are the focus of this chapter.

Mesh Representation in Unity

Unity manages geometry data through the **Mesh** class.

A model's shape is composed of triangles in 3D space, where each triangle is defined by 3 vertices. From the Unity documentation:

> In the Mesh class, all vertices are stored in a single array, and each triangle is specified by three integers corresponding to vertex array indices. Triangles are collected into an integer array,

grouped in threes from the beginning—elements 0, 1, 2 define the first triangle, elements 3, 4, 5 define the second, and so on.

Each vertex can have associated data: - **UV coordinates**: For texture mapping - **Normals**: For lighting calculations

Mental Model

Think of a Mesh as: - A bag of vertices (positions in 3D space) - A list of triangles (triplets of vertex indices) - Per-vertex attributes (UVs, normals, colors, tangents)

The triangles index into the vertex array—vertices can be shared across multiple triangles.

Quad: The Simplest Mesh

Let's start with the most basic shape: a **Quad** (4 vertices, 2 triangles forming a square).

```
0 -------- 1
|  \       |
|    \     |
|      \   |
3 -------- 2
```

The numbers represent vertex indices. Two triangles: (0,1,2) and (2,3,0).

Sample: Quad.cs

Step 1: Create Mesh instance

```
var mesh = new Mesh();
```

Step 2: Define vertex data

```
// Half size for centering the quad at origin
var hsize = size * 0.5f;

// Quad vertex positions
var vertices = new Vector3[] {
    new Vector3(-hsize,  hsize, 0f), // 0: top-left
    new Vector3( hsize,  hsize, 0f), // 1: top-right
    new Vector3( hsize, -hsize, 0f), // 2: bottom-right
    new Vector3(-hsize, -hsize, 0f)  // 3: bottom-left
};

// UV coordinates (texture mapping)
var uv = new Vector2[] {
    new Vector2(0f, 0f), // vertex 0
    new Vector2(1f, 0f), // vertex 1
    new Vector2(1f, 1f), // vertex 2
    new Vector2(0f, 1f)  // vertex 3
};

// Normals (all pointing toward camera, -Z direction)
var normals = new Vector3[] {
    new Vector3(0f, 0f, -1f),
    new Vector3(0f, 0f, -1f),
    new Vector3(0f, 0f, -1f),
```

```
    new Vector3(0f, 0f, -1f)
};
```

Step 3: Define triangles

```
// Triangle indices (3 indices per triangle)
var triangles = new int[] {
    0, 1, 2, // First triangle (top-right)
    2, 3, 0  // Second triangle (bottom-left)
};
```

> **Winding Order**
>
> Triangle vertex order matters! Unity uses clockwise winding for front-facing triangles. If you specify vertices counter-clockwise, the triangle faces away from the camera and may be culled.

Step 4: Assign to Mesh

```
mesh.vertices = vertices;
mesh.uv = uv;
mesh.normals = normals;
mesh.triangles = triangles;

// Calculate bounding box (needed for culling)
mesh.RecalculateBounds();

return mesh;
```

ProceduralModelingBase

The sample code uses a `ProceduralModelingBase` base class. When parameters change, it automatically regenerates the Mesh and applies it to the MeshFilter—providing instant visual feedback in the Editor.

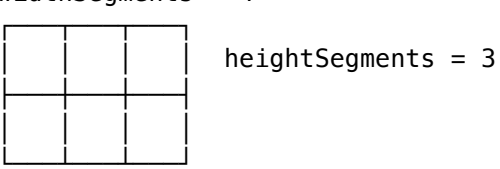
The `ProceduralModelingMaterial` enum lets you visualize UV coordinates and normals for debugging.

Primitive Shapes

Now let's build more complex primitives.

Plane

A **Plane** is a grid of Quads:



Parameters: - `widthSegments`: Vertices along width - `heightSegments`: Vertices along height - `width`: Total width - `height`: Total height

Sample: Plane.cs

Generate vertex grid:

```
var vertices = new List<Vector3>();
var uv = new List<Vector2>();
var normals = new List<Vector3>();

// Inverse of segment counts for calculating [0,1] ratios
var winv = 1f / (widthSegments - 1);
var hinv = 1f / (heightSegments - 1);

for(int y = 0; y < heightSegments; y++) {
    var ry = y * hinv;  // Row ratio [0,1]

    for(int x = 0; x < widthSegments; x++) {
        var rx = x * winv;  // Column ratio [0,1]

        vertices.Add(new Vector3(
            (rx - 0.5f) * width,   // Center on X
            0f,                     // Flat on Y
            (0.5f - ry) * height   // Center on Z
        ));
        uv.Add(new Vector2(rx, ry));
        normals.Add(new Vector3(0f, 1f, 0f));  // All point up
    }
}
```

Generate triangles:

```
var triangles = new List<int>();

for(int y = 0; y < heightSegments - 1; y++) {
    for(int x = 0; x < widthSegments - 1; x++) {
        int index = y * widthSegments + x;
        var a = index;
        var b = index + 1;
        var c = index + 1 + widthSegments;
        var d = index + widthSegments;

        // Two triangles per grid cell
        triangles.Add(a);
        triangles.Add(b);
        triangles.Add(c);

        triangles.Add(c);
        triangles.Add(d);
        triangles.Add(a);
    }
}
```

Index Calculation

For a 2D grid stored in a 1D array: `index = y * width + x`

This is a fundamental pattern you'll use constantly in graphics programming.

ParametricPlaneBase

By varying each vertex's Y coordinate based on UV position, you can create terrain:

```
protected override Mesh Build() {
    var mesh = base.Build();  // Generate base plane
    var vertices = mesh.vertices;

    var winv = 1f / (widthSegments - 1);
    var hinv = 1f / (heightSegments - 1);

    for(int y = 0; y < heightSegments; y++) {
        var ry = y * hinv;
        for(int x = 0; x < widthSegments; x++) {
            var rx = x * winv;
            int index = y * widthSegments + x;

            // Override Y with height function
            vertices[index].y = Depth(rx, ry);
        }
    }

    mesh.vertices = vertices;
    mesh.RecalculateBounds();
    mesh.RecalculateNormals();  // Auto-compute normals from geometry

    return mesh;
}
```

Subclasses implement `Depth(float u, float v)` to define height fields—mountains, terrain, waves, etc.

Cylinder

A **Cylinder** is a tube with circular cross-sections:

```
    ___
   /   \   <- top cap (optional)
  |     |
  |     |  <- side surface
  |     |
   \___/   <- bottom cap (optional)
```

Parameters: - `segments`: Smoothness of circle (7 = heptagon, higher = rounder) - `height`: Length of cylinder - `radius`: Radius of circle - `openEnded`: Whether to close the caps

Placing Vertices on a Circle

To place vertices evenly around a circle, use trigonometry:

```
for (int i = 0; i < segments; i++) {
    float ratio = (float)i / (segments - 1);  // [0, 1]
    float rad = ratio * Mathf.PI * 2;         // [0, 2π]

    float cos = Mathf.Cos(rad);
    float sin = Mathf.Sin(rad);
    float x = cos * radius;
```

```
    float z = sin * radius;
}
```

Why cos/sin?

On the unit circle: - `cos(θ)` gives the X coordinate - `sin(θ)` gives the Y coordinate (or Z in 3D)

Multiply by radius to scale the circle.

Sample: Cylinder.cs

Generate cap vertices:

```
void GenerateCap(
    int segments, float top, float bottom, float radius,
    List<Vector3> vertices, List<Vector2> uvs,
    List<Vector3> normals, bool side)
{
    for (int i = 0; i < segments; i++) {
        float ratio = (float)i / (segments - 1);
        float rad = ratio * PI2;

        float cos = Mathf.Cos(rad), sin = Mathf.Sin(rad);
        float x = cos * radius, z = sin * radius;

        Vector3 tp = new Vector3(x, top, z);
        Vector3 bp = new Vector3(x, bottom, z);

        vertices.Add(tp);  // Top ring vertex
        uvs.Add(new Vector2(ratio, 1f));

        vertices.Add(bp);  // Bottom ring vertex
        uvs.Add(new Vector2(ratio, 0f));

        if(side) {
            // Side normals point outward
            var normal = new Vector3(cos, 0f, sin);
            normals.Add(normal);
            normals.Add(normal);
        } else {
            // Cap normals point up/down
            normals.Add(new Vector3(0f, 1f, 0f));
            normals.Add(new Vector3(0f, -1f, 0f));
        }
    }
}
```

Why Separate Vertices for Caps?

Side surfaces and caps need different normals: - Sides: Normals point outward (radially) - Caps: Normals point up/down

If vertices were shared, normals would be averaged, creating unnatural lighting at edges. Always duplicate vertices where you need different normals (hard edges).

Build side triangles:

```
var len = (segments + 1) * 2;  // Total vertices in one ring pair

for (int i = 0; i < segments + 1; i++) {
    int idx = i * 2;
    int a = idx, b = idx + 1;
    int c = (idx + 2) % len, d = (idx + 3) % len;

    triangles.Add(a); triangles.Add(c); triangles.Add(b);
    triangles.Add(d); triangles.Add(b); triangles.Add(c);
}
```

Build caps (pizza-slice triangles from center):

```
// Center vertices
vertices.Add(new Vector3(0f, top, 0f));     // Top center
vertices.Add(new Vector3(0f, bottom, 0f));  // Bottom center

var it = vertices.Count - 2;  // Top center index
var ib = vertices.Count - 1;  // Bottom center index

// Top cap triangles
for (int i = 0; i < len; i += 2) {
    triangles.Add(it);
    triangles.Add((i + 2) % len + offset);
    triangles.Add(i + offset);
}

// Bottom cap triangles
for (int i = 1; i < len; i += 2) {
    triangles.Add(ib);
    triangles.Add(i + offset);
    triangles.Add((i + 2) % len + offset);
}
```

Tubular

A **Tubular** is a tube that follows a curve—like a pipe or branch:

Unlike a straight Cylinder, Tubular bends smoothly without twisting. This is essential for tree branches, tentacles, cables, and other organic shapes.

Tubular Structure

1. **Divide the curve** into segments
2. **At each segment**, place a ring of vertices (like a Cylinder cross-section)
3. **Connect adjacent rings** with triangles

Curves

The sample uses `CatmullRomCurve`, a spline that passes through all control points—intuitive for artists since the curve actually goes through the points you specify.

Key functions: - `GetPointAt(float t)`: Position at parameter t [0,1] - `GetTangentAt(float t)`: Direction at parameter t

Frenet Frames

To create a twist-free tube along a curve, we need three orthogonal vectors at each point:

- **Tangent (T)**: Direction along the curve
- **Normal (N)**: Perpendicular to tangent
- **Binormal (B)**: Perpendicular to both (N × T)

These three vectors form a **Frenet frame**—a local coordinate system that moves along the curve.

> **Why Frenet Frames?**
>
> To place vertices in a circle around the curve, we need to know "which way is up" and "which way is sideways" at each point. The normal and binormal give us these directions.
>
> `Circle vertex = CurvePoint + radius * (cos(θ) * N + sin(θ) * B)`

Sample: Tubular.cs

```
// Get Frenet frames along curve
var frames = curve.ComputeFrenetFrames(tubularSegments, closed);

// Generate vertices for each segment
for(int i = 0; i < tubularSegments; i++) {
    GenerateSegment(curve, frames, vertices, normals, tangents, i);
}

// Generate triangles connecting adjacent rings
for (int j = 1; j <= tubularSegments; j++) {
    for (int i = 1; i <= radialSegments; i++) {
        int a = (radialSegments + 1) * (j - 1) + (i - 1);
        int b = (radialSegments + 1) * j + (i - 1);
        int c = (radialSegments + 1) * j + i;
        int d = (radialSegments + 1) * (j - 1) + i;

        triangles.Add(a); triangles.Add(d); triangles.Add(b);
        triangles.Add(b); triangles.Add(d); triangles.Add(c);
    }
}
```

GenerateSegment function:

```
void GenerateSegment(CurveBase curve, List<FrenetFrame> frames,
    List<Vector3> vertices, List<Vector3> normals,
    List<Vector4> tangents, int index)
{
    var u = 1f * index / tubularSegments;
    var p = curve.GetPointAt(u);
    var fr = frames[index];

    var N = fr.Normal;
    var B = fr.Binormal;

    for(int j = 0; j <= radialSegments; j++) {
        float rad = 1f * j / radialSegments * PI2;

        float cos = Mathf.Cos(rad), sin = Mathf.Sin(rad);
        var v = (cos * N + sin * B).normalized;
```

```
            vertices.Add(p + radius * v);
            normals.Add(v);
            tangents.Add(new Vector4(fr.Tangent.x, fr.Tangent.y, fr.Tangent.z, 0f));
        }
    }
```

Complex Shapes: Trees

Plants are a classic application of procedural modeling. Unity even includes a Tree Editor, and tools like SpeedTree specialize in this.

L-Systems

L-Systems (Lindenmayer Systems) were proposed by botanist Aristid Lindenmayer in 1968 to describe plant structures.

L-Systems express **self-similarity**—the property where parts of an object resemble the whole at a smaller scale. Tree branches exhibit this: the branching pattern near the trunk is similar to branching patterns at the tips.

> **How L-Systems Work**
>
> 1. Start with an initial string (axiom)
> 2. Apply rewriting rules repeatedly
> 3. Each generation produces more complex results
>
> Example: - Axiom: `a` - Rules: `a → ab`, `b → a` - Generations: `a → ab → aba → abaab → abaababa → ...`

For graphics, symbols represent actions: - **Draw**: Move forward, drawing a line - **Turn Left**: Rotate left by θ degrees - **Turn Right**: Rotate right by θ degrees

Applying rules recursively generates tree-like branching patterns—this self-similarity is also called **fractal** structure.

ProceduralTree

The `ProceduralTree` class applies L-System concepts to generate 3D tree meshes.

Unlike the simple L-System example (fixed angles, binary branching), ProceduralTree uses randomness for: - Number of branches at each fork - Angles of branching - Branch lengths and radii

TreeData Class

`TreeData` contains parameters controlling tree shape:

Branching parameters: - `branchesMin/branchesMax`: Range for number of child branches - `growthAngleMin/growthAngleMax`: Range for branching angles - `growthAngleScale`: Increases angle variation toward branch tips

Mesh parameters: - `radialSegments`: Smoothness around branch circumference - `heightSegments`: Segments along branch length

TreeBranch Class

Each branch is a `TreeBranch` instance:

```
var root = new TreeBranch(generations, length, radius, data);
```

The constructor recursively creates child branches: - Each `TreeBranch` has a `List<TreeBranch> children` - Starting from root, you can traverse the entire tree

Branch Direction with Random Rotation

Each branch has tangent (direction), normal, and binormal vectors. When creating child branches:

```
// More angle variation toward branch tips
var scale = Mathf.Lerp(1f, data.growthAngleScale,
                       1f - 1f * generation / generations);

// Rotation around normal axis
var qn = Quaternion.AngleAxis(scale * data.GetRandomGrowthAngle(), normal);

// Rotation around binormal axis
var qb = Quaternion.AngleAxis(scale * data.GetRandomGrowthAngle(), binormal);

// Apply rotations to get new branch direction
this.to = from + (qn * qb) * tangent * length;
```

TreeSegment Class

Each branch is divided into segments (like Tubular):

```
public class TreeSegment {
    public FrenetFrame Frame { get { return frame; } }
    public Vector3 Position { get { return position; } }
    public float Radius { get { return radius; } }

    FrenetFrame frame;    // Direction vectors
    Vector3 position;     // Segment center
    float radius;         // Segment width
}
```

Building the Tree Mesh

The tree mesh combines all branches into one:

```
var root = new TreeBranch(generations, length, radius, data);

// Calculate total tree length for UV mapping
float maxLength = TraverseMaxLength(root);

// Traverse all branches recursively
Traverse(root, (branch) => {
    var offset = vertices.Count;

    // Generate vertices for this branch
    for(int i = 0, n = branch.Segments.Count; i < n; i++) {
        var segment = branch.Segments[i];
        var N = segment.Frame.Normal;
```

```
        var B = segment.Frame.Binormal;

        for(int j = 0; j <= data.radialSegments; j++) {
            float rad = 1f * j / data.radialSegments * PI2;
            float cos = Mathf.Cos(rad), sin = Mathf.Sin(rad);
            var normal = (cos * N + sin * B).normalized;

            vertices.Add(segment.Position + segment.Radius * normal);
            normals.Add(normal);
            // ... uvs, tangents
        }
    }

    // Generate triangles for this branch
    for (int j = 1; j <= data.heightSegments; j++) {
        for (int i = 1; i <= data.radialSegments; i++) {
            // ... quad indices with offset
        }
    }
});
```

> **Tree Modeling Goes Deeper**
>
> More sophisticated techniques consider: - Sunlight exposure (branches grow toward light) - Gravity effects (branches droop) - Wind response
>
> For further reading: “The Algorithmic Beauty of Plants” by Lindenmayer http://algorithmicbotany.org/papers/#abop

---

Application: Teddy (Sketch-Based Modeling)

Procedural modeling isn’t just for automated content—it enables interactive modeling tools.

Teddy, developed by Takeo Igarashi at the University of Tokyo, generates 3D models from 2D sketches:

1. User draws a 2D outline
2. Delaunay triangulation creates a 2D mesh
3. An inflation algorithm lifts the mesh into 3D

This technology powered “Rakugaki Oukoku” (2002, PlayStation 2), where players drew characters that became 3D fighters.

Teddy is available as a Unity asset: http://unitedy.info/

> **The Power of Procedural Techniques**
>
> With procedural modeling, you can build custom modeling tools—enabling content that evolves through user creativity.

Summary

Procedural modeling enables:

1. **Efficient model generation** with parametric control
2. **Interactive tools** that generate models from user input
3. **Runtime content** that adapts to gameplay

While this chapter focused on game/visualization applications, the techniques apply broadly: - Architecture and product design (Grasshopper/Rhinoceros) - Digital fabrication and 3D printing - Scientific visualization

Understanding procedural modeling opens possibilities wherever shape needs to be computed rather than manually created.

Key Takeaways

What You Should Remember

1. **Mesh = Vertices + Triangles**: Vertices are positions; triangles are triplets of vertex indices
2. **Winding order matters**: Clockwise = front-facing in Unity
3. **Duplicate vertices for hard edges**: Where normals differ, you need separate vertices
4. **Grid indexing**: `index = y * width + x` for 2D arrays in 1D storage
5. **Trigonometry for circles**: `(cos(θ) * r, sin(θ) * r)` places points on a circle
6. **Frenet frames**: Tangent + Normal + Binormal = local coordinate system along a curve
7. **L-Systems**: Recursive rewriting rules generate self-similar (fractal) structures
8. **RecalculateBounds()**: Always call after modifying vertices (needed for culling)
9. **RecalculateNormals()**: Auto-computes smooth normals from geometry

---

Shape Hierarchy

Shape	Structure	Key Concept
Quad	4 vertices, 2 triangles	Basic mesh building block
Plane	Grid of Quads	`y * width + x` indexing
Cylinder	Circular caps + side	Trigonometry for circles
Tubular	Cylinder along curve	Frenet frames
Tree	Recursive Tubulars	L-Systems, self-similarity

References

- The Algorithmic Beauty of Plants - http://algorithmicbotany.org/papers
- Teddy: A Sketching Interface for 3D Freeform Design - http://www-ui.is.s.u-tokyo.ac.jp/~takeo/papers/siggraph99.pdf
- Unity Mesh Documentation - https://docs.unity3d.com/Manual/AnatomyofaMesh.html

---

Next chapter: MultiPlane Projection!

Chapter 10: MultiPlane Perspective Projection

Original author: @fuqunaga Translation and annotations by Claude

---

Introduction

This chapter explains how to project imagery onto multiple surfaces (walls, floor) of a rectangular room using projectors, creating an immersive experience where viewers feel they're inside a CG world.

We'll cover the underlying theory of CG camera processing and its practical applications.

Sample project is in **Assets/RoomProjection** at: https://github.com/IndieVisualLab/UnityGraphicsProgramming

What You'll Learn

- How CG cameras work (coordinate transformations)
- How to match perspective across multiple cameras
- Deriving and modifying projection matrices
- Practical room-scale projection mapping

How CG Cameras Work

A CG camera performs **perspective projection**—transforming 3D models within a visible region into a 2D image.

This involves a sequence of coordinate system transformations:

```
Local Space → World Space → View Space → Clip Space → NDC → Screen Space
```

Coordinate System	Description
Local Space	Each model's origin at its own center
World Space	Shared origin for the entire scene
View Space	Camera at origin, looking down -Z
Clip Space	4D homogeneous coordinates (x,y,z,w) for clipping
NDC	Normalized Device Coordinates [-1,1] range
Screen Space	2D pixel coordinates on output display

Key Insight: Matrix Composition

Each transformation can be represented as a matrix. By multiplying matrices together in advance, we can perform multiple coordinate transformations with a single matrix-vector multiplication—a massive performance win on GPUs.

Matching Perspective Across Multiple Cameras

The View Frustum

A camera's **view frustum** is a truncated pyramid (frustum) that represents the 3D volume the camera can see: - **Apex**: Camera position - **Base**: The projection plane (screen) - **Sides**: Field of view boundaries

Connecting Frustums

When two camera frustums: 1. Share the same apex (camera position) 2. Have adjacent sides that touch

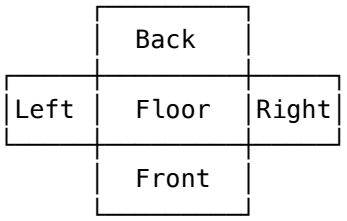
…their projected images will connect seamlessly with matching perspective!

Why This Works

Think of the frustum as a bundle of rays emanating from the camera. If two frustums share an apex and their boundaries touch, the rays form a continuous set—no gaps, no overlaps. The perspective is consistent because all rays originate from the same point.

Five-Camera Room Setup

By extending this to 5 cameras (4 walls + floor), with all frustums sharing one apex and touching at their edges, we can generate images for each room surface that create a cohesive immersive environment.



When viewed from the apex (the shared viewpoint), every direction shows perspective-correct imagery.

Why Not 6 Faces?

Theoretically possible, but the ceiling is often used for projector mounting. The sample assumes 5 faces (excluding ceiling).

Deriving the Projection Matrix

The **projection matrix** (Proj) transforms from View Space to Clip Space:

- **V**: Position in View Space
- **C**: Position in Clip Space

$$C = Proj \times V$$

To get NDC (Normalized Device Coordinates), divide by the w component:

$$NDC = \left(\frac{C_x}{C_w}, \frac{C_y}{C_w}, \frac{C_z}{C_w} \right)$$

The projection matrix is constructed so that $C_w = -V_z$ (negative because View Space looks down -Z).

The division by V_z creates the perspective effect—distant objects (larger |z|) get scaled down.

Building the Matrix

Let's define: - **N**: Top-right corner of the near clip plane - **F**: Top-right corner of the far clip plane

For X scaling:

We want the view to map to NDC range [-1, 1], and we know we'll divide by $C_w = -V_z$:

$$Proj[0,0] = \frac{N_z}{N_x}$$

For Y scaling (same logic):

$$Proj[1,1] = \frac{N_z}{N_y}$$

For Z (depth mapping):

This is trickier. The Z calculation is:

$$C_z = Proj[2,2] \times V_z + Proj[2,3] \times V_w \quad \text{(where } V_w = 1\text{)}$$

$$NDC_z = \frac{C_z}{C_w} \quad \text{(where } C_w = -V_z\text{)}$$

We want $N_z \to -1$ and $F_z \to 1$. Setting up equations:

$$-1 = \frac{1}{N_z}(a \cdot N_z + b)$$
$$1 = \frac{1}{F_z}(a \cdot F_z + b)$$

Solving this system:

$$Proj[2,2] = a = \frac{F_z + N_z}{F_z - N_z}$$
$$Proj[2,3] = b = \frac{-2F_zN_z}{F_z - N_z}$$

For W (perspective divide setup):

$$Proj[3,2] = -1$$

The Complete Projection Matrix

$$Proj = \begin{pmatrix} \frac{N_z}{N_x} & 0 & 0 & 0 \\ 0 & \frac{N_z}{N_y} & 0 & 0 \\ 0 & 0 & \frac{F_z+N_z}{F_z-N_z} & \frac{-2F_zN_z}{F_z-N_z} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Understanding the Matrix

- **Row 0**: X scaling (based on horizontal FOV)
- **Row 1**: Y scaling (based on vertical FOV)
- **Row 2**: Z remapping (near□ -1, far□ 1)
- **Row 3**: Perspective divide setup ($C_w = -V_z$)

Unity's Projection Matrix Gotcha

If you've worked with projection matrices in Unity shaders, some of this might seem off. Here's why:

Camera.projectionMatrix follows OpenGL conventions: - NDC z range: [-1, 1] - $C_w = -V_z$

However, Unity converts this to platform-specific formats when passing to shaders. Different platforms handle Z-buffer depth differently: - Some use [0, 1] range - Some reverse the direction

This is a common source of bugs. When working with projection matrices directly, use `GL.GetGPUProjectionMatrix()` to get the platform-correct version.

Manipulating the Frustum

Matching Projection Size to Room Faces

The frustum's base (projection plane) shape depends on: - **Field of View (fov)**: Vertical angle - **Aspect Ratio**: Width/height ratio

Unity's camera exposes FOV in the Inspector, but aspect ratio must be set via code.

Given the **face size** (wall dimensions) and **distance** (viewpoint to wall):

```
camera.aspect = faceSize.x / faceSize.y;
camera.fieldOfView = 2f * Mathf.Atan2(faceSize.y * 0.5f, distance)
                    * Mathf.Rad2Deg;
```

Breaking This Down

- `Atan2(opposite, adjacent)` gives the angle in radians
- We use half the height because FOV is measured from center
- Multiply by 2 to get full vertical FOV
- Convert to degrees for Unity's API

Lens Shift (Off-Center Projection)

What if the viewpoint isn't centered on the room? We need to shift the projection plane horizontally or vertically while keeping the same FOV.

This is called **lens shift**—equivalent to what physical projectors do when adjusting image position without moving the projector.

How it works in the projection matrix:

You might think to modify Proj[0,3] and Proj[1,3] (the translation components), but remember—we divide by C_w afterward. To shift in NDC space, modify Proj[0,2] and Proj[1,2]:

$$Proj = \begin{pmatrix} \frac{N_z}{N_x} & 0 & LensShift_x & 0 \\ 0 & \frac{N_z}{N_y} & LensShift_y & 0 \\ 0 & 0 & \frac{F_z+N_z}{F_z-N_z} & \frac{-2F_zN_z}{F_z-N_z} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Implementation:

```
// Convert position offset to NDC units [-1, 1]
var shift = new Vector2(
    positionOffset.x / faceSize.x,
```

```
    positionOffset.y / faceSize.y
) * 2f;

var projectionMatrix = camera.projectionMatrix;
projectionMatrix[0, 2] = shift.x;
projectionMatrix[1, 2] = shift.y;
camera.projectionMatrix = projectionMatrix;
```

> **Important Warning**
>
> Once you set `Camera.projectionMatrix` directly, changes to `Camera.fieldOfView` are ignored until you call `Camera.ResetProjectionMatrix()`. The camera remembers that you've overridden its projection.

Room Projection System

Putting it all together for an immersive room:

Given: - Rectangular room dimensions - Viewer position (tracked in real-time)

For each surface (4 walls + floor): 1. Calculate distance from viewer to surface 2. Set camera FOV and aspect to match surface dimensions 3. Apply lens shift based on viewer offset from surface center 4. Render to texture 5. Project texture onto physical surface

When the viewer stands at the tracked position, all surfaces show perspective-correct imagery—creating the illusion of being inside the CG world.

Practical Considerations

This technique creates a form of VR without a headset—surrounding the viewer with reactive imagery. However, some limitations exist:

Challenges: - No stereoscopic depth (both eyes see the same image) - Viewer can see the flat projection surfaces

Mitigations: - **Increase room size**: Greater distance reduces binocular disparity effects - **Control lighting**: Dark content minimizes surface reflections - **Use matte materials**: Avoid specular reflections that reveal the flat surface - **Track viewer accurately**: Perspective breaks if tracking is off

> **CAVE Systems**
>
> Professional installations called **CAVE** (Cave Automatic Virtual Environment) combine this multi-plane projection with stereoscopic displays for true 3D immersion. The viewer wears lightweight stereo glasses while surrounded by tracked projection surfaces.

Summary

This chapter demonstrated how understanding projection matrices enables creative camera configurations:

1. **Coordinate transformations** convert 3D scenes to 2D images
2. **View frustums** can be arranged to create seamless multi-screen displays
3. **Projection matrix manipulation** allows custom FOV and lens shift
4. **Room-scale projection** creates immersive environments

The math behind CG cameras isn't just academic—it's the foundation for practical applications from projection mapping to VR to architectural visualization.

Key Takeaways

What You Should Remember

1. **Coordinate pipeline**: Local World View Clip NDC Screen
2. **Projection matrix**: Transforms View Space to Clip Space, creating perspective
3. **Perspective divide**: Dividing by w ($-V_z$) makes distant objects smaller
4. **Matching frustums**: Same apex + touching edges = seamless perspective
5. **FOV calculation**: `2 * atan2(halfHeight, distance)` in radians
6. **Lens shift**: Modify Proj[0,2] and Proj[1,2], not [0,3] and [1,3]
7. **Platform differences**: Unity's projection matrix varies by platform (OpenGL vs D3D depth range)
8. **ResetProjectionMatrix()**: Required after manual projection matrix changes before FOV changes take effect

---

Projection Matrix Reference

Element	Purpose	Formula
Proj[0,0]	X scale	N_z/N_x
Proj[1,1]	Y scale	N_z/N_y
Proj[2,2]	Z remap (scale)	$(F_z + N_z)/(F_z - N_z)$
Proj[2,3]	Z remap (offset)	$-2F_zN_z/(F_z - N_z)$
Proj[3,2]	W setup	-1
Proj[0,2]	Lens shift X	User-defined
Proj[1,2]	Lens shift Y	User-defined

References

- Unity Projection Matrix Documentation - https://docs.unity3d.com/ScriptReference/Camera-projectionMatrix.html
- Platform Rendering Differences - https://docs.unity3d.com/Manual/SL-PlatformDifferences.html
- CAVE Systems - https://en.wikipedia.org/wiki/Cave_automatic_virtual_environment

---

This concludes Volume 1 of Unity Graphics Programming!